Northwind SMEL Scripts (Pauschalisiert)

1/16 northwind_pg1_to_pg2.smel_ps

```
-- SMEL Northwind: PostgreSQL V1 -> PostgreSQL V2 (Specific Operations Version)
-- Same-model Schema Evolution: Relational -> Relational
-- V2 Business Scenario: E-commerce platform modernization
--
-- Structural changes:
--   MERGE:  Denormalize categories into products (eliminate JOIN)
--   SPLIT:  Separate customer identity from contact info (CRM)

MIGRATION northwind_pg1_to_pg2:1.0
FROM RELATIONAL TO RELATIONAL
USING northwind_schema:1

-- ============================================
-- Meta V1 (after Reverse Engineering of northwind_postgresql.sql)
-- ============================================
-- categories (category_id [PK], category_name, description)
-- suppliers (supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- shippers (shipper_id [PK], company_name, phone)
-- employees (employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_co
-- customers (customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- products (product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->supplier
-- orders (order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_
-- order_details (order_id [PK,FK->orders], product_id [PK,FK->products], unit_price, quantity, discount)

-- ============================================
-- Phase 1: MERGE categories INTO products (structural)
-- Business: Eliminate JOIN for product catalog queries
-- ============================================
-- Before: categories(category_id, category_name, description)
--         products(..., category_id [FK->categories])
-- After:  products(..., category_name, description) -- categories removed
DELETE_PS CONSTRAINT products.category_id
DELETE_PS ATTRIBUTE products.category_id
DELETE_PS PRIMARY KEY category_id FROM categories
DELETE_PS ATTRIBUTE categories.category_id
MERGE_PS categories, products INTO products

-- ============================================
-- Phase 2: SPLIT customers -> customers + customer_contacts (structural)
-- Business: Separate company identity from contact person info
-- ============================================
-- Before: customers(customer_id, company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, countr
-- After:  customers(customer_id, company_name, street, city, region, postal_code, country)
--         customer_contacts(customer_id, contact_name, contact_title, phone, fax)
SPLIT_PS customers INTO customers:customer_id, company_name, street, city, region, postal_code, country; customer_contacts:custome
ADD_PS KEY customer_id TO customers
ADD_PS KEY customer_contacts.contact_id AS String
ADD_PS CONSTRAINT customer_contacts.customer_id REFERENCES customers(customer_id) WITH CARDINALITY ONE_TO_ONE

-- ============================================
-- Phase 3: suppliers -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE company_name TO name IN suppliers
ADD_PS ATTRIBUTE website TO suppliers WITH TYPE String

-- ============================================
-- Phase 4: shippers -> carriers (RENAME entity + field)
-- ============================================
RENAME_PS ENTITY shippers TO carriers
RENAME_PS ATTRIBUTE company_name TO carrier_name IN carriers

-- ============================================
-- Phase 5: employees -- simplify + ADD
-- ============================================
DELETE_PS ATTRIBUTE employees.birth_date
DELETE_PS ATTRIBUTE employees.notes
ADD_PS ATTRIBUTE email TO employees WITH TYPE String

-- ============================================
-- Phase 6: customer_contacts -- ADD email
-- ============================================
ADD_PS ATTRIBUTE email TO customer_contacts WITH TYPE String

-- ============================================
-- Phase 7: products -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE product_name TO name IN products
ADD_PS ATTRIBUTE sku TO products WITH TYPE String

-- ============================================
-- Phase 8: orders -- RENAME + ADD
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- ============================================
RENAME_PS ATTRIBUTE freight TO shipping_cost IN orders
ADD_PS ATTRIBUTE status TO orders WITH TYPE String

-- ============================================
-- Phase 9: order_details -- RENAME
-- ============================================
RENAME_PS ATTRIBUTE discount TO discount_rate IN order_details

-- ============================================
-- Final Meta V2 (8 entities):
-- products:           { product_id [PK], name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK], cat
-- suppliers:          { supplier_id [PK], name, contact_name, contact_title, phone, fax, street, city, postal_code, country, webs
-- carriers:           { shipper_id [PK], carrier_name, phone }
-- employees:          { employee_id [PK], last_name, first_name, title, hire_date, email, phone, street, city, region, postal_cod
-- customers:          { customer_id [PK], company_name, street, city, postal_code, country }
-- customer_contacts:  { contact_id [PK], customer_id [FK], contact_name, contact_title, phone, fax, email }
-- orders:             { order_id [PK], order_date, required_date, shipped_date, shipping_cost, ship_name, ship_address, ship_city
-- order_details:      { order_id [PK,FK], product_id [PK,FK], unit_price, quantity, discount_rate }
-- ============================================

-- ============================================
-- Operation Summary (21 steps):
--   MERGE: 1              SPLIT: 1
--   DELETE_CONSTRAINT: 1  ADD_CONSTRAINT: 1
--   DELETE_PRIMARY_KEY: 1 ADD_PRIMARY_KEY: 2
--   DELETE_ATTRIBUTE: 4   ADD_ATTRIBUTE: 5
--   RENAME_ATTRIBUTE: 5   RENAME_ENTITY: 1
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

2/16 northwind_mongo1_to_mongo2.smel_ps

```
-- SMEL Northwind: MongoDB V1 -> MongoDB V2 (Specific Operations Version)
-- Same-model Schema Evolution: Document -> Document
-- V2 Business Scenario: E-commerce platform modernization
--
-- Structural changes:
--   FLATTEN:   Flatten category sub-object into product (denormalize, reduce L3->L2)
--   UNFLATTEN: Group customer contact fields into contacts sub-object (reorganize)

MIGRATION northwind_mongo1_to_mongo2:1.0
FROM DOCUMENT TO DOCUMENT
USING northwind_schema:1

-- ============================================
-- Meta V1 (after Reverse Engineering of northwind_mongodb.json)
-- ============================================
-- orders (root) { _id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }
--   embedded: customer { company_name, contact_name, contact_title, phone, fax }
--     embedded: address { street, city, region, postal_code, country }           [L2]
--   embedded: employee { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to }
--     embedded: address { street, city, region, postal_code, country }           [L2]
--   embedded: shipper { company_name, phone }
--   embedded: details[] (ONE_TO_MANY)
--     embedded: product { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
--       embedded: category { category_name, description }               [L3]
--       embedded: supplier { company_name, contact_name, contact_title, phone, fax }
--         embedded: address { street, city, region, postal_code, country }      [L4]

-- ============================================
-- Phase 1: FLATTEN category into product (structural -- reduce nesting)
-- Document equivalent of MERGE: L3 -> L2, category sub-object disappears
-- ============================================
-- Before: product { ..., category { category_name, description }, supplier { ... } }
-- After:  product { ..., category_category_name, category_description, supplier { ... } }
FLATTEN_PS orders.details.product.category
RENAME_PS ATTRIBUTE category_category_name TO category_name IN product
RENAME_PS ATTRIBUTE category_description TO description IN product
-- Result: product { product_name, unit_price, ..., category_name, description, supplier { ... } }

-- ============================================
-- Phase 2: UNFLATTEN customer contact fields into contacts sub-object (structural)
-- Document equivalent of SPLIT: group contact person fields into sub-object
-- ============================================
-- Before: customer { company_name, contact_name, contact_title, phone, fax, address { ... } }
-- After:  customer { company_name, address { ... }, contacts { contact_name, contact_title, phone, fax } }
UNFLATTEN_PS customer:contact_name, contact_title, phone, fax AS contacts

-- ============================================
-- Phase 3: suppliers -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE company_name TO name IN supplier
ADD_PS ATTRIBUTE website TO supplier WITH TYPE String

-- ============================================
-- Phase 4: shipper -- RENAME
-- ============================================
RENAME_PS ATTRIBUTE company_name TO carrier_name IN shipper

-- ============================================
-- Phase 5: employee -- simplify + ADD
-- ============================================
DELETE_PS ATTRIBUTE employee.birth_date
DELETE_PS ATTRIBUTE employee.notes
ADD_PS ATTRIBUTE email TO employee WITH TYPE String

-- ============================================
-- Phase 6: contacts -- ADD email
-- ============================================
ADD_PS ATTRIBUTE email TO contacts WITH TYPE String

-- ============================================
-- Phase 7: product -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE product_name TO name IN product
ADD_PS ATTRIBUTE sku TO product WITH TYPE String

-- ============================================
-- Phase 8: orders -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE freight TO shipping_cost IN orders
ADD_PS ATTRIBUTE status TO orders WITH TYPE String
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- ============================================
-- Phase 9: details (order line items) -- RENAME
-- ============================================
RENAME_PS ATTRIBUTE discount TO discount_rate IN details

-- ============================================
-- Final Meta V2 (4-level nesting, max L3 after FLATTEN):
-- orders { _id [PK], order_date, required_date, shipped_date, shipping_cost, ship_name, ship_city, status }
--   customer { company_name, address { street, city, region, postal_code, country },
--              contacts { contact_name, contact_title, phone, fax, email } }
--   employee { last_name, first_name, title, hire_date, email, phone, reports_to,
--              address { street, city, region, postal_code, country } }
--   shipper { carrier_name, phone }
--   details[] {
--     unit_price, quantity, discount_rate,
--     product { name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--               category_name, description, sku,
--               supplier { name, contact_name, contact_title, phone, fax, website,
--                          address { street, city, region, postal_code, country } } }
--   }
-- ============================================

-- ============================================
-- Operation Summary (16 steps):
--   FLATTEN: 1             UNFLATTEN: 1
--   RENAME_ATTRIBUTE: 7    ADD_ATTRIBUTE: 5
--   DELETE_ATTRIBUTE: 2
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

3/16 northwind_graph1_to_graph2.smel_ps

```
-- SMEL Northwind: Neo4j V1 -> Neo4j V2 (Specific Operations Version)
-- Same-model Schema Evolution: Graph -> Graph
-- V2 Business Scenario: E-commerce platform modernization
--
-- Structural changes:
--   MERGE:         Combine categories into products node (denormalize graph)
--   SPLIT:         Separate customers into customers + contact nodes
--   DELETE_RELTYPE + ADD_RELTYPE: Restructure graph edges
--   ADD_LABEL:     Multi-label node
--   RENAME_RELTYPE: Rename relationship types

MIGRATION northwind_graph1_to_graph2:1.0
FROM GRAPH TO GRAPH
USING northwind_schema:1

-- ============================================
-- Meta V1 (after Reverse Engineering of northwind_neo4j.cypher)
-- ============================================
-- 7 Nodes:
--   categories { category_id, category_name, description }
--   suppliers { supplier_id, company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
--   shippers { shipper_id, company_name, phone }
--   employees { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code, countr
--   customers { customer_id, company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
--   products { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
--   orders { order_id, order_date, required_date, shipped_date, freight, ship_name, ship_city }
-- 7 Relationships:
--   SUPPLIES (suppliers -> products), PART_OF (products -> categories)
--   PURCHASED (customers -> orders), SOLD (employees -> orders)
--   SHIPPED_VIA (orders -> shippers), REPORTS_TO (employees -> employees)
--   CONTAINS (orders -> products) [props: unit_price, quantity, discount]

-- ============================================
-- Phase 1: MERGE categories INTO products (structural)
-- Graph equivalent of denormalization: eliminate PART_OF relationship
-- ============================================
-- Before: products { product_id, product_name, ... } -[PART_OF]-> categories { category_id, category_name, description }
-- After:  products { product_id, product_name, ..., category_name, description }
DELETE_PS RELTYPE PART_OF
DELETE_PS ATTRIBUTE categories.category_id
MERGE_PS categories, products INTO products

-- ============================================
-- Phase 2: SPLIT customers -> customers + contact (structural)
-- Graph equivalent of CRM separation: create contact node with relationship
-- ============================================
-- Before: customers { customer_id, company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- After:  customers { customer_id, company_name, street, city, postal_code, country }
--         contact { customer_id, contact_name, contact_title, phone, fax }
--         customers -[HAS_CONTACT]-> contact
SPLIT_PS customers INTO customers:customer_id, company_name, street, city, region, postal_code, country; contact:customer_id, cont
ADD_PS KEY customer_id TO customers
ADD_PS KEY contact.contact_id AS String
ADD_PS RELTYPE HAS_CONTACT BETWEEN customers AND contact

-- ============================================
-- Phase 3: suppliers -- RENAME + ADD + ADD_LABEL
-- ============================================
RENAME_PS ATTRIBUTE company_name TO name IN suppliers
ADD_PS ATTRIBUTE website TO suppliers WITH TYPE String
ADD_PS LABEL Vendor TO suppliers

-- ============================================
-- Phase 4: shippers -- RENAME
-- ============================================
RENAME_PS ATTRIBUTE company_name TO carrier_name IN shippers

-- ============================================
-- Phase 5: employees -- simplify + ADD
-- ============================================
DELETE_PS ATTRIBUTE employees.birth_date
DELETE_PS ATTRIBUTE employees.notes
ADD_PS ATTRIBUTE email TO employees WITH TYPE String

-- ============================================
-- Phase 6: contact -- ADD email
-- ============================================
ADD_PS ATTRIBUTE email TO contact WITH TYPE String

-- ============================================
-- Phase 7: products -- RENAME + ADD
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- ============================================
RENAME_PS ATTRIBUTE product_name TO name IN products
ADD_PS ATTRIBUTE sku TO products WITH TYPE String

-- ============================================
-- Phase 8: orders -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE freight TO shipping_cost IN orders
ADD_PS ATTRIBUTE status TO orders WITH TYPE String

-- ============================================
-- Phase 9: RENAME relationship types
-- ============================================
RENAME_PS RELTYPE SUPPLIES TO PROVIDES
RENAME_PS RELTYPE SHIPPED_VIA TO DELIVERS

-- ============================================
-- Final Meta V2 (7 nodes, 7 relationships):
-- products:   { product_id [PK], name, unit_price, units_in_stock, discontinued, quantity_per_unit, category_name, description, s
-- suppliers:  { supplier_id [PK], name, contact_name, contact_title, phone, fax, street, city, postal_code, country, website } +V
-- shippers:   { shipper_id [PK], carrier_name, phone }
-- employees:  { employee_id [PK], last_name, first_name, title, hire_date, email, phone, street, city, region, postal_code, count
-- customers:  { customer_id [PK], company_name, street, city, postal_code, country }
-- contact:    { contact_id [PK], customer_id, contact_name, contact_title, phone, fax, email }
-- orders:     { order_id [PK], order_date, required_date, shipped_date, shipping_cost, ship_name, ship_city, status }
-- [categories MERGED into products, PART_OF deleted]
-- [customers SPLIT -> customers + contact, HAS_CONTACT added]
-- Relationships: PROVIDES, HAS_CONTACT, PURCHASED, SOLD, DELIVERS, REPORTS_TO, CONTAINS
-- ============================================

-- ============================================
-- Operation Summary (22 steps):
--   MERGE: 1              SPLIT: 1
--   DELETE_RELTYPE: 1     ADD_RELTYPE: 2
--   ADD_PRIMARY_KEY: 2    DELETE_ATTRIBUTE: 3
--   ADD_ATTRIBUTE: 5      RENAME_ATTRIBUTE: 5
--   ADD_LABEL: 1          RENAME_RELTYPE: 2
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

4/16 northwind_cass1_to_cass2.smel_ps

```
-- SMEL Northwind: Cassandra V1 -> Cassandra V2 (Specific Operations Version)
-- Same-model Schema Evolution: Columnar -> Columnar
-- V2 Business Scenario: E-commerce platform modernization
--
-- Structural changes:
--   MERGE:               Denormalize categories into products table
--   SPLIT:               Separate customer identity from contact info
--   Repartition orders:  Change partition key from customer_id to order_date

MIGRATION northwind_cass1_to_cass2:1.0
FROM COLUMNAR TO COLUMNAR
USING northwind_schema:1

-- ============================================
-- Meta V1 (after Reverse Engineering of northwind_cassandra.cql)
-- ============================================
-- categories (category_id [PARTITION], category_name, description)
-- suppliers (supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, c
-- shippers (shipper_id [PARTITION], company_name, phone)
-- employees (employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, po
-- customers (customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, c
-- products (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_stock,
-- orders (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight, ship_nam
-- order_details (order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount)

-- ============================================
-- Phase 1: MERGE categories INTO products (structural + repartition)
-- Cassandra: categories partition key consumed, products repartitioned
-- ============================================
-- Before: products PK((category_id, supplier_id), product_id)
-- After:  products PK((product_id)) with category_name, description added
DELETE_PS PARTITION KEY (category_id, supplier_id) FROM products
DELETE_PS CLUSTERING KEY product_id FROM products
DELETE_PS ATTRIBUTE products.category_id
DELETE_PS PARTITION KEY category_id FROM categories
DELETE_PS ATTRIBUTE categories.category_id
MERGE_PS categories, products INTO products
ADD_PS PARTITION KEY product_id TO products

-- ============================================
-- Phase 2: SPLIT customers -> customers + customer_contacts (structural)
-- ============================================
-- Before: customers PK((customer_id), ...)
-- After:  customers PK((customer_id))
--         customer_contacts PK((customer_id), contact_id)
SPLIT_PS customers INTO customers:customer_id, company_name, street, city, region, postal_code, country; customer_contacts:custome
ADD_PS PARTITION KEY customer_id TO customers
ADD_PS PARTITION KEY customer_id TO customer_contacts
ADD_PS CLUSTERING KEY contact_id TO customer_contacts

-- ============================================
-- Phase 3: Repartition orders (query optimization)
-- Change from customer-based to date-based partitioning
-- ============================================
-- Before: orders PK((customer_id), order_date, order_id)
-- After:  orders PK((order_date), customer_id, order_id)
DELETE_PS PARTITION KEY customer_id FROM orders
DELETE_PS CLUSTERING KEY order_date FROM orders
DELETE_PS CLUSTERING KEY order_id FROM orders
ADD_PS PARTITION KEY order_date TO orders
ADD_PS CLUSTERING KEY customer_id TO orders
ADD_PS CLUSTERING KEY order_id TO orders

-- ============================================
-- Phase 4: suppliers -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE company_name TO name IN suppliers
ADD_PS ATTRIBUTE website TO suppliers WITH TYPE String

-- ============================================
-- Phase 5: shippers -> carriers (RENAME entity + field)
-- ============================================
RENAME_PS ENTITY shippers TO carriers
RENAME_PS ATTRIBUTE company_name TO carrier_name IN carriers

-- ============================================
-- Phase 6: employees -- simplify + ADD
-- ============================================
DELETE_PS ATTRIBUTE employees.birth_date
DELETE_PS ATTRIBUTE employees.notes
ADD_PS ATTRIBUTE email TO employees WITH TYPE String
```

```
-- ============================================
-- Phase 7: customer_contacts -- ADD email
-- ============================================
ADD_PS ATTRIBUTE email TO customer_contacts WITH TYPE String

-- ============================================
-- Phase 8: products -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE product_name TO name IN products
ADD_PS ATTRIBUTE sku TO products WITH TYPE String

-- ============================================
-- Phase 9: orders -- RENAME + ADD
-- ============================================
RENAME_PS ATTRIBUTE freight TO shipping_cost IN orders
ADD_PS ATTRIBUTE status TO orders WITH TYPE String

-- ============================================
-- Phase 10: order_details -- RENAME
-- ============================================
RENAME_PS ATTRIBUTE discount TO discount_rate IN order_details

-- ============================================
-- Final Meta V2 (8 entities):
-- products:           { product_id [PARTITION], name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id, c
-- suppliers:          { supplier_id [PARTITION], name, contact_name, contact_title, phone, fax, street, city, postal_code, countr
-- carriers:           { shipper_id [PARTITION], carrier_name, phone }
-- employees:          { employee_id [PARTITION], last_name, first_name, title, hire_date, email, phone, street, city, region, pos
-- customers:          { customer_id [PARTITION], company_name, street, city, postal_code, country }
-- customer_contacts:  { customer_id [PARTITION], contact_id [CLUSTERING], contact_name, contact_title, phone, fax, email }
-- orders:             { order_date [PARTITION], customer_id [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, shi
-- order_details:      { order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount_rate }
-- ============================================

-- ============================================
-- Operation Summary (28 steps):
--   MERGE: 1             SPLIT: 1
--   DELETE_PARTITION_KEY: 3  ADD_PARTITION_KEY: 3
--   DELETE_CLUSTERING_KEY: 3 ADD_CLUSTERING_KEY: 3
--   DELETE_ATTRIBUTE: 5   ADD_ATTRIBUTE: 5
--   RENAME_ATTRIBUTE: 5   RENAME_ENTITY: 1
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

5/16 northwind_pg_to_mongo.smel_ps

```
-- SMEL Northwind: PostgreSQL -> MongoDB (Specific Operations Version)
-- Normalized Tables (3NF) -> Orders Document (multi-level nested objects + arrays)
-- Direction: Normalized (3NF) -> Denormalized (Embedded)
--
-- Operation Semantics:
--   NEST:      Embed separate table into parent as nested object (reverse of UNNEST)
--   UNFLATTEN: Combine flat fields into nested object (reverse of FLATTEN)

MIGRATION northwind_pg_to_mongo:1.0
FROM RELATIONAL TO DOCUMENT
USING northwind_schema:1

-- ============================================
-- Source: PostgreSQL 3NF Tables (8 tables)
-- ============================================
-- categories (category_id [PK], category_name, description)
-- suppliers (supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- shippers (shipper_id [PK], company_name, phone)
-- employees (employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_co
-- customers (customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- products (product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->supplier
-- orders (order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_
-- order_details (order_id [PK,FK->orders], product_id [PK,FK->products], unit_price, quantity, discount)

-- ============================================
-- Target: MongoDB Orders Document
-- ============================================
-- {
--   _id: "ORD001",
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Step 1: UNFLATTEN address fields (3 entities have flat address -> sub-objects)
-- ============================================
-- Reverse of: FLATTEN suppliers.address / customers.address / employees.address
UNFLATTEN_PS suppliers:street, city, region, postal_code, country AS address
UNFLATTEN_PS customers:street, city, region, postal_code, country AS address
UNFLATTEN_PS employees:street, city, region, postal_code, country AS address
UNFLATTEN_PS orders:ship_address, ship_city, ship_region, ship_postal_code, ship_country AS ship_destination
-- Result:
-- suppliers: { supplier_id, company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, cou
-- customers: { customer_id, company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, cou
-- employees: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city,

-- ============================================
-- Step 2: NEST categories into products (Level 3 -- deepest embedding first)
-- ============================================
-- Reverse of: UNNEST products.category:category_name, description AS categories
NEST_PS categories:category_name, description IN products.category WHERE products.category_id = categories.category_id WITH DELETI
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id,
--             category{category_name, description} }

-- ============================================
-- Step 3: NEST suppliers into products (Level 3 -- with nested address)
-- ============================================
-- Reverse of: UNNEST products.supplier:company_name, contact_name, contact_title, phone, fax, address{...} AS suppliers
NEST_PS suppliers:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country} IN pr
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             category{category_name, description},
--             supplier{company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country}

-- ============================================
-- Step 4: NEST products into order_details (Level 2)
-- ============================================
-- Reverse of: UNNEST order_details.product:product_name, unit_price, ... AS products
```

Northwind SMEL Scripts (Pauschalisiert)

```
NEST_PS products:product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, category{category_name, description},
-- Result:
-- order_details: { order_id, unit_price, quantity, discount,
--                  product{product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                          category{category_name, description},
--                          supplier{company_name, contact_name, contact_title, phone, fax, address{...}}} }

-- ============================================
-- Step 5: NEST shippers into orders (Level 1)
-- ============================================
-- Reverse of: UNNEST orders.shipper:company_name, phone AS shippers
NEST_PS shippers:company_name, phone IN orders.shipper WHERE orders.shipper_id = shippers.shipper_id WITH DELETION
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer_id, employee_id, shipper{company_name, phone} }

-- ============================================
-- Step 6: NEST employees into orders (Level 1 -- with address + self-ref)
-- ============================================
-- Reverse of: UNNEST orders.employee:last_name, first_name, ... AS employees
NEST_PS employees:last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city, region, pos
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer_id, shipper{...},
--           employee{last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{...}} }

-- ============================================
-- Step 7: NEST customers into orders (Level 1 -- with address)
-- ============================================
-- Reverse of: UNNEST orders.customer:company_name, contact_name, ... AS customers
NEST_PS customers:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country} IN or
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           shipper{...}, employee{...},
--           customer{company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country}}

-- ============================================
-- Step 8: NEST order_details into orders (Level 1 -- as array)
-- ============================================
-- Reverse of: UNNEST orders.details:unit_price, quantity, discount, product{...} AS order_details
NEST_PS order_details:unit_price, quantity, discount, product{product_name, unit_price, units_in_stock, discontinued, quantity_per
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer{...}, employee{...}, shipper{...},
--           details[{unit_price, quantity, discount, product{...}}] }

-- ============================================
-- Step 9: Rename PK for MongoDB convention
-- ============================================
-- Reverse of: RENAME_ATTRIBUTE _id TO order_id IN orders
RENAME_PS ATTRIBUTE order_id TO _id IN orders
-- Final Result:
-- orders: {
--   _id,
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Operation Summary (12 steps):
--   UNFLATTEN: 4       (address sub-objects for suppliers, customers, employees + ship_destination for orders)
--   NEST: 7            (categories->products, suppliers->products, products->order_details,
--                       shippers->orders, employees->orders, customers->orders, order_details->orders)
--   RENAME_ATTRIBUTE: 1 (order_id -> _id)
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

6/16 northwind_mongo_to_pg.smel_ps

```
-- SMEL Northwind: MongoDB -> PostgreSQL (Pauschalisiert Operations Version)
-- Orders Document (multi-level nested objects + arrays) -> Normalized Tables (3NF)
-- Direction: Denormalized (Embedded) -> Normalized (3NF)
--
-- Operation Semantics:
--   UNNEST:  Extract nested object to separate table (normalization)
--   FLATTEN: Flatten nested object fields into same table (reduce depth by 1)

MIGRATION northwind_mongo_to_pg:1.0
FROM DOCUMENT TO RELATIONAL
USING northwind_schema:1

-- ============================================
-- Source: MongoDB Orders Document
-- ============================================
-- {
--   _id: "ORD001",
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Target: PostgreSQL 3NF Tables (8 tables)
-- ============================================
-- categories (category_id [PK], category_name, description)
-- suppliers (supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- shippers (shipper_id [PK], company_name, phone)
-- employees (employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_co
-- customers (customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- products (product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->supplier
-- orders (order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_
-- order_details (order_id [PK,FK->orders], product_id [PK,FK->products], unit_price, quantity, discount)

-- ============================================
-- Step 1: Rename PK from MongoDB convention
-- ============================================
RENAME_PS ATTRIBUTE _id TO order_id IN orders
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer{...}, employee{...}, shipper{...}, details[{...}] }

-- ============================================
-- Step 2: UNNEST details[] -> order_details (array extraction)
-- ============================================
UNNEST_PS orders.details:unit_price, quantity, discount, product{product_name, unit_price, units_in_stock, discontinued, quantity_
-- Result:
-- orders: { order_id, ..., ship_destination{...}, customer{...}, employee{...}, shipper{...} }
-- order_details: { order_id, unit_price, quantity, discount, product{..., category{...}, supplier{...}} }

-- ============================================
-- Step 3: UNNEST customer -> customers + FLATTEN address
-- ============================================
UNNEST_PS orders.customer:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, countr
ADD_PS KEY customers.customer_id AS String
DELETE_PS ATTRIBUTE customers.order_id
ADD_PS ATTRIBUTE customer_id TO orders WITH TYPE String
ADD_PS CONSTRAINT orders.customer_id REFERENCES customers(customer_id) WITH CARDINALITY ONE_TO_MANY
-- FLATTEN address into customers
FLATTEN_PS customers.address
RENAME_PS ATTRIBUTE address_street TO street IN customers
RENAME_PS ATTRIBUTE address_city TO city IN customers
RENAME_PS ATTRIBUTE address_region TO region IN customers
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN customers
RENAME_PS ATTRIBUTE address_country TO country IN customers
-- Result:
-- orders: { order_id, ..., customer_id [FK->customers], employee{...}, shipper{...} }
-- customers: { customer_id, company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country }
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- ============================================
-- Step 4: UNNEST employee -> employees + FLATTEN address + self-ref
-- ============================================
UNNEST_PS orders.employee:last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city, reg
ADD_PS KEY employees.employee_id AS String
DELETE_PS ATTRIBUTE employees.order_id
ADD_PS ATTRIBUTE employee_id TO orders WITH TYPE String
ADD_PS CONSTRAINT orders.employee_id REFERENCES employees(employee_id) WITH CARDINALITY ONE_TO_MANY
ADD_PS CONSTRAINT employees.reports_to REFERENCES employees(employee_id) WITH CARDINALITY ZERO_TO_ONE
-- FLATTEN address into employees
FLATTEN_PS employees.address
RENAME_PS ATTRIBUTE address_street TO street IN employees
RENAME_PS ATTRIBUTE address_city TO city IN employees
RENAME_PS ATTRIBUTE address_region TO region IN employees
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN employees
RENAME_PS ATTRIBUTE address_country TO country IN employees
-- Result:
-- orders: { order_id, ..., customer_id [FK], employee_id [FK], shipper{...} }
-- employees: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_code,

-- ============================================
-- Step 5: UNNEST shipper -> shippers
-- ============================================
UNNEST_PS orders.shipper:company_name, phone AS shippers WITH orders.order_id TO shippers.order_id
ADD_PS KEY shippers.shipper_id AS String
DELETE_PS ATTRIBUTE shippers.order_id
ADD_PS ATTRIBUTE shipper_id TO orders WITH TYPE String
ADD_PS CONSTRAINT orders.shipper_id REFERENCES shippers(shipper_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination{...}, customer_id [FK], employee_id [FK], shipper_id [FK] }
-- shippers: { shipper_id, company_name, phone }

-- ============================================
-- Step 6: UNNEST product -> products (from order_details, Level 2)
-- ============================================
UNNEST_PS order_details.product:product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, category{category_name,
ADD_PS KEY products.product_id AS String
DELETE_PS ATTRIBUTE products.order_id
ADD_PS ATTRIBUTE product_id TO order_details WITH TYPE String
ADD_PS CONSTRAINT order_details.product_id REFERENCES products(product_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- order_details: { order_id, product_id [FK], unit_price, quantity, discount }
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             category{category_name, description}, supplier{..., address{...}} }

-- ============================================
-- Step 7: UNNEST supplier -> suppliers + FLATTEN address (from products, Level 3)
-- ============================================
UNNEST_PS products.supplier:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, coun
ADD_PS KEY suppliers.supplier_id AS String
DELETE_PS ATTRIBUTE suppliers.product_id
ADD_PS ATTRIBUTE supplier_id TO products WITH TYPE String
ADD_PS CONSTRAINT products.supplier_id REFERENCES suppliers(supplier_id) WITH CARDINALITY ONE_TO_MANY
-- FLATTEN address into suppliers
FLATTEN_PS suppliers.address
RENAME_PS ATTRIBUTE address_street TO street IN suppliers
RENAME_PS ATTRIBUTE address_city TO city IN suppliers
RENAME_PS ATTRIBUTE address_region TO region IN suppliers
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN suppliers
RENAME_PS ATTRIBUTE address_country TO country IN suppliers
-- Result:
-- products: { product_id, ..., supplier_id [FK->suppliers], category{...} }
-- suppliers: { supplier_id, company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country }

-- ============================================
-- Step 8: UNNEST category -> categories (from products, Level 3)
-- ============================================
UNNEST_PS products.category:category_name, description AS categories WITH products.product_id TO categories.product_id
ADD_PS KEY categories.category_id AS String
DELETE_PS ATTRIBUTE categories.product_id
ADD_PS ATTRIBUTE category_id TO products WITH TYPE String
ADD_PS CONSTRAINT products.category_id REFERENCES categories(category_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             supplier_id [FK->suppliers], category_id [FK->categories] }
-- categories: { category_id, category_name, description }

-- ============================================
-- Step 9: FLATTEN orders.ship_destination
-- ============================================
FLATTEN_PS orders.ship_destination
```

```
RENAME_PS ATTRIBUTE ship_destination_ship_address TO ship_address IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_city TO ship_city IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_region TO ship_region IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_postal_code TO ship_postal_code IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_country TO ship_country IN orders
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_address, ship_city, ship_region, ship_postal_code, ship_country,
--   customer_id [FK], employee_id [FK], shipper_id [FK] }

-- ============================================
-- Step 10: Add composite PK for order_details + FK to orders
-- ============================================
ADD_PS CONSTRAINT order_details.order_id REFERENCES orders(order_id) WITH CARDINALITY ONE_TO_MANY
ADD_PS KEY (order_id, product_id) TO order_details

-- ============================================
-- Final Result (8 tables):
-- categories:    { category_id [PK], category_name, description }
-- suppliers:     { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, co
-- shippers:      { shipper_id [PK], company_name, phone }
-- employees:     { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, pos
-- customers:     { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, co
-- products:      { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->s
-- orders:        { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_regi
-- order_details: { order_id [PK,FK->orders], product_id [PK,FK->products], unit_price, quantity, discount }
-- ============================================

-- ============================================
-- Operation Summary (42 steps):
--   RENAME_ATTRIBUTE: 13  (1 PK rename + 12 FLATTEN renames)
--   UNNEST: 7             (details, customer, employee, shipper, product, supplier, category)
--   FLATTEN: 4            (customers.address, employees.address, suppliers.address, orders.ship_destination)
--   ADD_PRIMARY_KEY: 7    (customers, employees, shippers, products, suppliers, categories, order_details composite)
--   DELETE_ATTRIBUTE: 6   (carry-field cleanup for customers, employees, shippers, products, suppliers, categories)
--   ADD_ATTRIBUTE: 6      (FK fields: customer_id, employee_id, shipper_id, product_id, supplier_id, category_id)
--   ADD_CONSTRAINT: 9     (7 FK constraints + 1 self-ref + 1 order_details->orders)
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

7/16 northwind_pg_to_neo4j.smel_ps

```
-- SMEL: PostgreSQL -> Neo4j (Specific Operations Version)
-- Cross-model Schema Migration: Relational -> Graph
-- Direction: northwind_postgresql.sql -> northwind_neo4j.cypher
--
-- Pipeline: Source PG -> Reverse Engineering -> Meta V1
--           -> SMEL Operations (this script) -> Meta V2
--           -> Forward Engineering -> Target Neo4j
--
-- Operation Semantics:
--   DELETE_CONSTRAINT: Remove FK constraint (prepare FK for graph edge conversion)
--   DELETE_ATTRIBUTE:  Remove FK attribute (no longer needed as graph edge replaces it)
--   ADD_RELTYPE:       Add relationship type between nodes (graph edge)
--   DELETE_PRIMARY_KEY: Remove composite PK before structural TRANSFORM
--   TRANSFORM:         Convert entity (table) to relationship (graph edge)

MIGRATION northwind_pg_to_neo4j:1.0
FROM RELATIONAL TO GRAPH
USING northwind_schema:1

-- ============================================
-- Source: PostgreSQL 3NF Tables (8 tables, 8 FKs)
-- ============================================
-- categories (category_id [PK], category_name, description)
-- suppliers (supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- shippers (shipper_id [PK], company_name, phone)
-- employees (employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_co
-- customers (customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- products (product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->supplier
-- orders (order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_
-- order_details (order_id [PK,FK->orders], product_id [PK,FK->products], unit_price, quantity, discount)

-- ============================================
-- Target: Neo4j Graph (7 nodes, 7 relationships)
-- ============================================
-- Nodes: categories, suppliers, shippers, employees, customers, products, orders
-- Relationships:
--   REPORTS_TO  (employees -> employees)       [self-reference]
--   SUPPLIES    (suppliers -> products)
--   PART_OF     (products -> categories)
--   PURCHASED   (customers -> orders)
--   SOLD        (employees -> orders)
--   SHIPPED_VIA (orders -> shippers)
--   CONTAINS    (orders -> products)           [with properties: unit_price, quantity, discount]

-- ============================================
-- Step 1: employees -- self-reference FK -> REPORTS_TO relationship
-- ============================================
-- The reports_to FK column references employees(employee_id), creating a
-- hierarchical self-reference. In graph model, this becomes an edge.
DELETE_PS CONSTRAINT employees.reports_to
DELETE_PS ATTRIBUTE employees.reports_to
ADD_PS RELTYPE REPORTS_TO BETWEEN employees AND employees
-- Result:
-- employees: { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code, co
-- + REPORTS_TO relationship (employees -> employees)

-- ============================================
-- Step 2: products -- supplier FK -> SUPPLIES relationship
-- ============================================
-- supplier_id FK becomes a SUPPLIES edge from suppliers to products.
DELETE_PS CONSTRAINT products.supplier_id
DELETE_PS ATTRIBUTE products.supplier_id
ADD_PS RELTYPE SUPPLIES BETWEEN suppliers AND products
-- Result:
-- products: { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, category_id }
-- + SUPPLIES relationship (suppliers -> products)

-- ============================================
-- Step 3: products -- category FK -> PART_OF relationship
-- ============================================
-- category_id FK becomes a PART_OF edge from products to categories.
DELETE_PS CONSTRAINT products.category_id
DELETE_PS ATTRIBUTE products.category_id
ADD_PS RELTYPE PART_OF BETWEEN products AND categories
-- Result:
-- products: { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }

-- ============================================
-- Step 4: orders -- customer FK -> PURCHASED relationship
-- ============================================
-- customer_id FK becomes a PURCHASED edge from customers to orders.
```

Northwind SMEL Scripts (Pauschalisiert)

```
DELETE_PS CONSTRAINT orders.customer_id
DELETE_PS ATTRIBUTE orders.customer_id
ADD_PS RELTYPE PURCHASED BETWEEN customers AND orders
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, shi
-- + PURCHASED relationship (customers -> orders)

-- ============================================
-- Step 5: orders -- employee FK -> SOLD relationship
-- ============================================
-- employee_id FK becomes a SOLD edge from employees to orders.
DELETE_PS CONSTRAINT orders.employee_id
DELETE_PS ATTRIBUTE orders.employee_id
ADD_PS RELTYPE SOLD BETWEEN employees AND orders
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, shi
-- + SOLD relationship (employees -> orders)

-- ============================================
-- Step 6: orders -- shipper FK -> SHIPPED_VIA relationship
-- ============================================
-- shipper_id FK becomes a SHIPPED_VIA edge from orders to shippers.
DELETE_PS CONSTRAINT orders.shipper_id
DELETE_PS ATTRIBUTE orders.shipper_id
ADD_PS RELTYPE SHIPPED_VIA BETWEEN orders AND shippers
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }

-- ============================================
-- Step 7: order_details -- junction table -> CONTAINS relationship with properties
-- ============================================
-- order_details is a junction table linking orders and products.
-- In graph model, it becomes a CONTAINS relationship with properties
-- (unit_price, quantity, discount). The FK columns and composite PK
-- are removed, and the remaining attributes become relationship properties.
DELETE_PS CONSTRAINT order_details.order_id
DELETE_PS CONSTRAINT order_details.product_id
DELETE_PS PRIMARY KEY (order_id, product_id) FROM order_details
DELETE_PS ATTRIBUTE order_details.order_id
DELETE_PS ATTRIBUTE order_details.product_id
TRANSFORM_PS order_details TO RELATIONSHIP BETWEEN orders AND products
RENAME_PS RELTYPE order_details TO CONTAINS
-- Result:
-- order_details table removed; CONTAINS relationship created:
-- CONTAINS (orders -> products) with properties: { unit_price, quantity, discount }
-- Note: TRANSFORM creates relationship with entity name "order_details",
--       then RENAME_RELTYPE renames it to match Neo4j convention "CONTAINS".

-- ============================================
-- Final Meta V2 (7 nodes, 7 relationship types):
-- ============================================
-- categories:  { category_id [PK], category_name, description }
-- suppliers:   { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- shippers:    { shipper_id [PK], company_name, phone }
-- employees:   { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code,
-- customers:   { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- products:    { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
-- orders:      { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }
-- REPORTS_TO  (employees -> employees)
-- SUPPLIES    (suppliers -> products)
-- PART_OF     (products -> categories)
-- PURCHASED   (customers -> orders)
-- SOLD        (employees -> orders)
-- SHIPPED_VIA (orders -> shippers)
-- CONTAINS    (orders -> products) with properties { unit_price, quantity, discount }
-- [order_details TRANSFORMED to CONTAINS relationship]
-- ============================================

-- ============================================
-- Operation Summary (25 steps, 6 operation types):
--   DELETE_CONSTRAINT:  8    DELETE_ATTRIBUTE:  8
--   ADD_RELTYPE:        6    DELETE_PRIMARY_KEY: 1
--   TRANSFORM:          1    RENAME_RELTYPE:    1
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

8/16 northwind_neo4j_to_pg.smel_ps

```
-- SMEL: Neo4j -> PostgreSQL (Specific Operations Version)
-- Cross-model Schema Migration: Graph -> Relational
-- Direction: northwind_neo4j.cypher -> northwind_postgresql.sql
--
-- Pipeline: Source Neo4j -> Reverse Engineering -> Meta V1
--           -> SMEL Operations (this script) -> Meta V2
--           -> Forward Engineering -> Target PG
--
-- Operation Semantics:
--   TRANSFORM:       Convert relationship (graph edge) to entity (table)
--   RENAME_ENTITY:   Rename entity after TRANSFORM (relationship name -> table name)
--   ADD_ATTRIBUTE:   Add FK attribute to entity (graph edge replaced by FK column)
--   ADD_CONSTRAINT:  Add FK constraint (re-establish relational references)
--   ADD_PRIMARY_KEY: Add composite PK to junction table
--   DELETE_RELTYPE:  Remove relationship type (graph edge no longer needed)
-- Note: Neo4j source already uses lowercase entity names (categories, suppliers, etc.)

MIGRATION northwind_neo4j_to_pg:1.0
FROM GRAPH TO RELATIONAL
USING northwind_schema:1

-- ============================================
-- Source: Neo4j Graph (7 nodes, 7 relationships)
-- ============================================
-- Nodes:
--   categories:  { category_id [PK], category_name, description }
--   suppliers:   { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
--   shippers:    { shipper_id [PK], company_name, phone }
--   employees:   { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code
--   customers:   { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
--   products:    { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
--   orders:      { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }
-- Relationships:
--   REPORTS_TO  (employees -> employees)
--   SUPPLIES    (suppliers -> products)
--   PART_OF     (products -> categories)
--   PURCHASED   (customers -> orders)
--   SOLD        (employees -> orders)
--   SHIPPED_VIA (orders -> shippers)
--   CONTAINS    (orders -> products) with properties { unit_price, quantity, discount }

-- ============================================
-- Target: PostgreSQL 3NF Tables (8 tables, 8 FKs)
-- ============================================
-- categories (category_id [PK], category_name, description)
-- suppliers (supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- shippers (shipper_id [PK], company_name, phone)
-- employees (employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_co
-- customers (customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
-- products (product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->supplier
-- orders (order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_
-- order_details (order_id [PK,FK->orders], product_id [PK,FK->products], unit_price, quantity, discount)

-- ============================================
-- Step 1: CONTAINS relationship -> order_details table
-- ============================================
-- The CONTAINS relationship carries properties (unit_price, quantity, discount).
-- TRANSFORM converts it to an entity, preserving those properties.
-- Then rename from relationship name to table name, add FK columns and composite PK.
TRANSFORM_PS CONTAINS TO ENTITY
RENAME_PS ENTITY CONTAINS TO order_details
ADD_PS ATTRIBUTE order_id TO order_details WITH TYPE String
ADD_PS ATTRIBUTE product_id TO order_details WITH TYPE String
ADD_PS CONSTRAINT order_details.order_id REFERENCES orders(order_id) WITH CARDINALITY ONE_TO_MANY
ADD_PS CONSTRAINT order_details.product_id REFERENCES products(product_id) WITH CARDINALITY ONE_TO_MANY
ADD_PS KEY (order_id, product_id) TO order_details
-- Result:
-- order_details: { order_id [PK,FK], product_id [PK,FK], unit_price, quantity, discount }

-- ============================================
-- Step 2: orders -- PURCHASED relationship -> customer_id FK
-- ============================================
-- PURCHASED (customers -> orders) becomes a customer_id FK column in orders.
DELETE_PS RELTYPE PURCHASED
ADD_PS ATTRIBUTE customer_id TO orders WITH TYPE String
ADD_PS CONSTRAINT orders.customer_id REFERENCES customers(customer_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, shi

-- ============================================
-- Step 3: orders -- SOLD relationship -> employee_id FK
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- ============================================
-- SOLD (employees -> orders) becomes an employee_id FK column in orders.
DELETE_PS RELTYPE SOLD
ADD_PS ATTRIBUTE employee_id TO orders WITH TYPE String
ADD_PS CONSTRAINT orders.employee_id REFERENCES employees(employee_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- orders: { ..., employee_id [FK] }

-- ============================================
-- Step 4: orders -- SHIPPED_VIA relationship -> shipper_id FK
-- ============================================
-- SHIPPED_VIA (orders -> shippers) becomes a shipper_id FK column in orders.
DELETE_PS RELTYPE SHIPPED_VIA
ADD_PS ATTRIBUTE shipper_id TO orders WITH TYPE String
ADD_PS CONSTRAINT orders.shipper_id REFERENCES shippers(shipper_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, shi

-- ============================================
-- Step 5: products -- SUPPLIES relationship -> supplier_id FK
-- ============================================
-- SUPPLIES (suppliers -> products) becomes a supplier_id FK column in products.
DELETE_PS RELTYPE SUPPLIES
ADD_PS ATTRIBUTE supplier_id TO products WITH TYPE String
ADD_PS CONSTRAINT products.supplier_id REFERENCES suppliers(supplier_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- products: { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK] }

-- ============================================
-- Step 6: products -- PART_OF relationship -> category_id FK
-- ============================================
-- PART_OF (products -> categories) becomes a category_id FK column in products.
DELETE_PS RELTYPE PART_OF
ADD_PS ATTRIBUTE category_id TO products WITH TYPE String
ADD_PS CONSTRAINT products.category_id REFERENCES categories(category_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- products: { ..., supplier_id [FK], category_id [FK] }

-- ============================================
-- Step 7: employees -- REPORTS_TO relationship -> reports_to FK (self-reference)
-- ============================================
-- REPORTS_TO (employees -> employees) becomes a reports_to FK column
-- referencing the same employees table (self-reference with ZERO_TO_ONE
-- cardinality since not every employee has a manager).
DELETE_PS RELTYPE REPORTS_TO
ADD_PS ATTRIBUTE reports_to TO employees WITH TYPE String
ADD_PS CONSTRAINT employees.reports_to REFERENCES employees(employee_id) WITH CARDINALITY ZERO_TO_ONE
-- Result:
-- employees: { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_

-- ============================================
-- Final Meta V2 (8 tables, 8 FK constraints):
-- ============================================
-- categories:     { category_id [PK], category_name, description }
-- suppliers:      { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- shippers:       { shipper_id [PK], company_name, phone }
-- employees:      { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, po
-- customers:      { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- products:       { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->
-- orders:         { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_reg
-- order_details:  { order_id [PK,FK->orders], product_id [PK,FK->products], unit_price, quantity, discount }
-- [CONTAINS TRANSFORMED to order_details entity]
-- ============================================

-- ============================================
-- Operation Summary (25 steps, 6 operation types):
--   RENAME_ENTITY:   1    ADD_ATTRIBUTE:   8
--   ADD_CONSTRAINT:  8    DELETE_RELTYPE:  6
--   ADD_PRIMARY_KEY: 1    TRANSFORM:       1
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

9/16 northwind_pg_to_cass.smel_ps

```
-- SMEL Northwind: PostgreSQL -> Cassandra (Cross-model Migration)
-- Relational (3NF with FKs) -> Columnar (query-driven partition/clustering keys)
-- Direction: northwind_postgresql.sql -> northwind_cassandra.cql
--
-- Pipeline: Source PG -> Reverse Engineering -> Meta V1 (Relational)
--           -> SMEL Operations (this script) -> Meta V2 (Columnar)
--           -> Forward Engineering -> Target Cassandra
--
-- Operation Semantics:
--   DELETE_CONSTRAINT:    Remove FK references (Cassandra has no FKs)
--   DELETE_PRIMARY_KEY:   Remove relational PK before adding partition/clustering keys
--   ADD_PARTITION_KEY:    Define partition key for query-driven data distribution
--   ADD_CLUSTERING_KEY:   Define clustering key for sort order within partition

MIGRATION northwind_pg_to_cass:1.0
FROM RELATIONAL TO COLUMNAR
USING northwind_schema:1

-- ============================================
-- Source: PostgreSQL 3NF Tables (8 tables)
-- ============================================
-- categories     (category_id [PK], category_name, description)
-- suppliers      (supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, cou
-- shippers       (shipper_id [PK], company_name, phone)
-- employees      (employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, post
-- customers      (customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, cou
-- products       (product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->su
-- orders         (order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_regio
-- order_details  (order_id [PK, FK->orders], product_id [PK, FK->products], unit_price, quantity, discount)

-- ============================================
-- Target: Cassandra Wide-column Tables (8 tables)
-- ============================================
-- categories     (category_id [PARTITION])
-- suppliers      (supplier_id [PARTITION])
-- shippers       (shipper_id [PARTITION])
-- employees      (employee_id [PARTITION], reports_to as plain column)
-- customers      (customer_id [PARTITION])
-- products       (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], ...)
-- orders         (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], ...)
-- order_details  (order_id [PARTITION], product_id [PARTITION], ...)

-- ============================================
-- Step 1: Remove self-reference FK from employees
-- ============================================
-- employees.reports_to references employees(employee_id)
-- Cassandra has no FK constraints; reports_to becomes a plain column
DELETE_PS CONSTRAINT employees.reports_to
-- Result:
-- employees (employee_id [PK], ..., reports_to) -- FK removed, field retained

-- ============================================
-- Step 2: Remove FK constraints from products
-- ============================================
-- products has 2 FKs: supplier_id -> suppliers, category_id -> categories
DELETE_PS CONSTRAINT products.supplier_id
DELETE_PS CONSTRAINT products.category_id
-- Result:
-- products (product_id [PK], ..., supplier_id, category_id) -- FKs removed, fields retained

-- ============================================
-- Step 3: Restructure products keys (PK -> partition + clustering)
-- ============================================
-- Relational: product_id is PK
-- Cassandra: partition by category_id (query products by category), cluster by product_id
DELETE_PS PRIMARY KEY product_id FROM products
ADD_PS PARTITION KEY (category_id, supplier_id) TO products
ADD_PS CLUSTERING KEY product_id TO products
-- Result:
-- products (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_stock,

-- ============================================
-- Step 4: Remove FK constraints from orders
-- ============================================
-- orders has 3 FKs: customer_id -> customers, employee_id -> employees, shipper_id -> shippers
DELETE_PS CONSTRAINT orders.customer_id
DELETE_PS CONSTRAINT orders.employee_id
DELETE_PS CONSTRAINT orders.shipper_id
-- Result:
-- orders (order_id [PK], ..., customer_id, employee_id, shipper_id) -- FKs removed
```

```
-- ============================================
-- Step 5: Restructure orders keys (PK -> partition + clustering)
-- ============================================
-- Relational: order_id is PK
-- Cassandra: partition by customer_id (query orders by customer), cluster by order_date, order_id
DELETE_PS PRIMARY KEY order_id FROM orders
ADD_PS PARTITION KEY customer_id TO orders
ADD_PS CLUSTERING KEY order_date TO orders
ADD_PS CLUSTERING KEY order_id TO orders
-- Result:
-- orders (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight, ship_nam

-- ============================================
-- Step 6: Remove FK constraints from order_details
-- ============================================
-- order_details has 2 FKs: order_id -> orders, product_id -> products
DELETE_PS CONSTRAINT order_details.order_id
DELETE_PS CONSTRAINT order_details.product_id
-- Result:
-- order_details (order_id [PK], product_id [PK], ...) -- FKs removed, composite PK remains

-- ============================================
-- Step 7: Restructure order_details keys (composite PK -> partition + clustering)
-- ============================================
-- Relational: composite PK (order_id, product_id)
-- Cassandra: partition by order_id (query details by order), cluster by product_id
DELETE_PS PRIMARY KEY (order_id, product_id) FROM order_details
ADD_PS PARTITION KEY (order_id, product_id) TO order_details
-- Result:
-- order_details (order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount)

-- ============================================
-- Simple tables: categories, suppliers, shippers, customers
-- ============================================
-- These tables have simple PKs and no outbound FKs.
-- The EntityKind auto-conversion (TABLE -> WIDE_COLUMN_TABLE) handles the
-- PK -> PARTITION KEY transformation automatically.
-- No explicit SMEL operations needed.

-- ============================================
-- Final Target (8 wide-column tables):
-- categories:     { category_id [PARTITION], category_name, description }
-- suppliers:      { supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, co
-- shippers:       { shipper_id [PARTITION], company_name, phone }
-- employees:      { employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, reg
-- customers:      { customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, co
-- products:       { category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in
-- orders:         { customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight
-- order_details:  { order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount }
-- ============================================

-- ============================================
-- Operation Summary (16 steps):
--   DELETE_CONSTRAINT:    8    DELETE_PRIMARY_KEY: 3
--   ADD_PARTITION_KEY:    3    ADD_CLUSTERING_KEY: 3
--   EntityKind auto-convert: TABLE -> WIDE_COLUMN_TABLE (all 8 tables)
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

10/16 northwind_cass_to_pg.smel_ps

```
-- SMEL Northwind: Cassandra -> PostgreSQL (Cross-model Migration)
-- Columnar (query-driven partition/clustering keys) -> Relational (3NF with FKs)
-- Direction: northwind_cassandra.cql -> northwind_postgresql.sql
--
-- Pipeline: Source Cassandra -> Reverse Engineering -> Meta V1 (Columnar)
--           -> SMEL Operations (this script) -> Meta V2 (Relational)
--           -> Forward Engineering -> Target PostgreSQL
--
-- Operation Semantics:
--   DELETE_PARTITION_KEY:   Remove Cassandra partition key before adding relational PK
--   DELETE_CLUSTERING_KEY:  Remove Cassandra clustering key
--   ADD_PRIMARY_KEY:        Add relational primary key (simple or composite)
--   ADD_CONSTRAINT:         Add FK references (relational referential integrity)

MIGRATION northwind_cass_to_pg:1.0
FROM COLUMNAR TO RELATIONAL
USING northwind_schema:1

-- ============================================
-- Source: Cassandra Wide-column Tables (8 tables)
-- ============================================
-- categories     (category_id [PARTITION], category_name, description)
-- suppliers      (supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- shippers       (shipper_id [PARTITION], company_name, phone)
-- employees      (employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, regio
-- customers      (customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- products       (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_s
-- orders         (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight,
-- order_details  (order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount)

-- ============================================
-- Target: PostgreSQL 3NF Tables (8 tables)
-- ============================================
-- categories     (category_id [PK])
-- suppliers      (supplier_id [PK])
-- shippers       (shipper_id [PK])
-- employees      (employee_id [PK], reports_to [FK->employees])
-- customers      (customer_id [PK])
-- products       (product_id [PK], supplier_id [FK->suppliers], category_id [FK->categories])
-- orders         (order_id [PK], customer_id [FK->customers], employee_id [FK->employees], shipper_id [FK->shippers])
-- order_details  (order_id [PK, FK->orders], product_id [PK, FK->products])

-- ============================================
-- Step 1: Restructure products keys (partition + clustering -> PK)
-- ============================================
-- Cassandra: category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING]
-- Relational: product_id [PK]
DELETE_PS PARTITION KEY (category_id, supplier_id) FROM products
DELETE_PS CLUSTERING KEY product_id FROM products
ADD_PS KEY product_id TO products
-- Result:
-- products (product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id, category_id)

-- ============================================
-- Step 2: Add FK constraints to products
-- ============================================
-- Restore referential integrity: supplier_id -> suppliers, category_id -> categories
ADD_PS CONSTRAINT products.supplier_id REFERENCES suppliers(supplier_id) WITH CARDINALITY ONE_TO_MANY
ADD_PS CONSTRAINT products.category_id REFERENCES categories(category_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- products (product_id [PK], ..., supplier_id [FK->suppliers], category_id [FK->categories])

-- ============================================
-- Step 3: Restructure orders keys (partition + clustering -> PK)
-- ============================================
-- Cassandra: customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING]
-- Relational: order_id [PK]
DELETE_PS PARTITION KEY customer_id FROM orders
DELETE_PS CLUSTERING KEY order_date FROM orders
DELETE_PS CLUSTERING KEY order_id FROM orders
ADD_PS KEY order_id TO orders
-- Result:
-- orders (order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_

-- ============================================
-- Step 4: Add FK constraints to orders
-- ============================================
-- Restore referential integrity: customer_id, employee_id, shipper_id
ADD_PS CONSTRAINT orders.customer_id REFERENCES customers(customer_id) WITH CARDINALITY ONE_TO_MANY
ADD_PS CONSTRAINT orders.employee_id REFERENCES employees(employee_id) WITH CARDINALITY ONE_TO_MANY
ADD_PS CONSTRAINT orders.shipper_id REFERENCES shippers(shipper_id) WITH CARDINALITY ONE_TO_MANY
```

```
-- Result:
-- orders (order_id [PK], ..., customer_id [FK->customers], employee_id [FK->employees], shipper_id [FK->shippers])

-- ============================================
-- Step 5: Restructure order_details keys (partition + clustering -> composite PK)
-- ============================================
-- Cassandra: order_id [PARTITION], product_id [PARTITION]
-- Relational: composite PK (order_id, product_id)
DELETE_PS PARTITION KEY (order_id, product_id) FROM order_details
ADD_PS KEY (order_id, product_id) TO order_details
-- Result:
-- order_details (order_id [PK], product_id [PK], unit_price, quantity, discount)

-- ============================================
-- Step 6: Add FK constraints to order_details
-- ============================================
-- Restore referential integrity: order_id -> orders, product_id -> products
ADD_PS CONSTRAINT order_details.order_id REFERENCES orders(order_id) WITH CARDINALITY ONE_TO_MANY
ADD_PS CONSTRAINT order_details.product_id REFERENCES products(product_id) WITH CARDINALITY ONE_TO_MANY
-- Result:
-- order_details (order_id [PK, FK->orders], product_id [PK, FK->products], unit_price, quantity, discount)

-- ============================================
-- Step 7: Add self-reference FK to employees
-- ============================================
-- employees.reports_to references employees(employee_id)
-- ZERO_TO_ONE: an employee may or may not have a manager
ADD_PS CONSTRAINT employees.reports_to REFERENCES employees(employee_id) WITH CARDINALITY ZERO_TO_ONE
-- Result:
-- employees (employee_id [PK], ..., reports_to [FK->employees])

-- ============================================
-- Simple tables: categories, suppliers, shippers, customers
-- ============================================
-- These tables have simple partition keys and no outbound references.
-- The EntityKind auto-conversion (WIDE_COLUMN_TABLE -> TABLE) handles the
-- PARTITION KEY -> PK transformation automatically.
-- No explicit SMEL operations needed.

-- ============================================
-- Final Target (8 relational tables):
-- categories:     { category_id [PK], category_name, description }
-- suppliers:      { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- shippers:       { shipper_id [PK], company_name, phone }
-- employees:      { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, po
-- customers:      { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- products:       { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id [FK->
-- orders:         { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_reg
-- order_details:  { order_id [PK, FK->orders], product_id [PK, FK->products], unit_price, quantity, discount }
-- ============================================

-- ============================================
-- Operation Summary (16 steps):
--   DELETE_PARTITION_KEY:  3    DELETE_CLUSTERING_KEY: 3
--   ADD_PRIMARY_KEY:       3    ADD_CONSTRAINT:        8
--   EntityKind auto-convert: WIDE_COLUMN_TABLE -> TABLE (all 8 tables)
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

11/16 northwind_mongo_to_neo4j.smel_ps

```
-- SMEL Northwind: MongoDB -> Neo4j (Specific Operations Version)
-- Document (Embedded 4-level nesting) -> Graph (Nodes + Relationships)
-- Direction: Denormalized (Embedded) -> Graph (Nodes + Edges)
--
-- Pipeline: Source MongoDB -> Reverse Engineering -> Meta V1 (Document)
--           -> SMEL Operations (this script) -> Meta V2 (Graph)
--           -> Forward Engineering -> Target Neo4j
--
-- Operation Semantics:
--   UNNEST:           Extract nested object to separate entity
--   FLATTEN:          Flatten nested object fields into parent (reduce depth by 1)
--   RENAME_ATTRIBUTE: Rename a field within an entity
--   ADD_PRIMARY_KEY:  Add primary key to entity
--   DELETE_ATTRIBUTE: Remove attribute from entity
--   ADD_RELTYPE:      Add relationship type between nodes
--   TRANSFORM:        Convert entity to relationship (or vice versa)

MIGRATION northwind_mongo_to_neo4j:1.0
FROM DOCUMENT TO GRAPH
USING northwind_schema:1

-- ============================================
-- Source: MongoDB Orders Document (1 root, 4-level nesting)
-- ============================================
-- {
--   _id: "ORD001",
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Target: Neo4j Graph (7 nodes, 7 relationships)
-- ============================================
-- Nodes:
--   categories (category_id, category_name, description)
--   suppliers (supplier_id, company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
--   shippers (shipper_id, company_name, phone)
--   employees (employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_code,
--   customers (customer_id, company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
--   products (product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit)
--   orders (order_id, order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_pos
-- Relationships:
--   SUPPLIES (suppliers -> products), PART_OF (products -> categories)
--   PURCHASED (customers -> orders), SOLD (employees -> orders), SHIPPED_VIA (orders -> shippers)
--   REPORTS_TO (employees -> employees)
--   CONTAINS (orders -> products) with properties: unit_price, quantity, discount

-- ============================================
-- Step 1: Rename primary key (_id -> order_id)
-- ============================================
RENAME_PS ATTRIBUTE _id TO order_id IN orders
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer{...}, employee{...}, shipper{...}, details[{...}] }

-- ============================================
-- Step 2: UNNEST details array -> order_details entity
-- ============================================
UNNEST_PS orders.details:unit_price, quantity, discount, product{product_name, unit_price, units_in_stock, discontinued, quantity_
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer{...}, employee{...}, shipper{...} }
-- order_details: { order_id, unit_price, quantity, discount, product{...} }

-- ============================================
-- Step 3: UNNEST customer -> customers entity
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

```
UNNEST_PS orders.customer:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, countr
DELETE_PS ATTRIBUTE customers.order_id
ADD_PS KEY customers.customer_id AS String
ADD_PS ATTRIBUTE customer_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id, ..., customer_id, employee{...}, shipper{...} }
-- customers: { customer_id, company_name, contact_name, contact_title, phone, fax, address{...} }

-- ============================================
-- Step 4: UNNEST employee -> employees entity
-- ============================================
UNNEST_PS orders.employee:last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city, reg
DELETE_PS ATTRIBUTE employees.order_id
ADD_PS KEY employees.employee_id AS String
ADD_PS ATTRIBUTE employee_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id, ..., customer_id, employee_id, shipper{...} }
-- employees: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{...} }

-- ============================================
-- Step 5: UNNEST shipper -> shippers entity
-- ============================================
UNNEST_PS orders.shipper:company_name, phone AS shippers WITH orders.order_id TO shippers.order_id
DELETE_PS ATTRIBUTE shippers.order_id
ADD_PS KEY shippers.shipper_id AS String
ADD_PS ATTRIBUTE shipper_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id, ..., customer_id, employee_id, shipper_id }
-- shippers: { shipper_id, company_name, phone }

-- ============================================
-- Step 6: UNNEST product -> products entity (from order_details)
-- ============================================
UNNEST_PS order_details.product:product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, category{category_name,
DELETE_PS ATTRIBUTE products.order_id
ADD_PS KEY products.product_id AS String
ADD_PS ATTRIBUTE product_id TO order_details WITH TYPE String
-- Result:
-- order_details: { order_id, product_id, unit_price, quantity, discount }
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             category{...}, supplier{...} }

-- ============================================
-- Step 7: UNNEST supplier -> suppliers entity (from products)
-- ============================================
UNNEST_PS products.supplier:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, coun
DELETE_PS ATTRIBUTE suppliers.product_id
ADD_PS KEY suppliers.supplier_id AS String
ADD_PS ATTRIBUTE supplier_id TO products WITH TYPE String
-- Result:
-- products: { product_id, ..., supplier_id, category{...} }
-- suppliers: { supplier_id, company_name, contact_name, contact_title, phone, fax, address{...} }

-- ============================================
-- Step 8: UNNEST category -> categories entity (from products)
-- ============================================
UNNEST_PS products.category:category_name, description AS categories WITH products.product_id TO categories.product_id
DELETE_PS ATTRIBUTE categories.product_id
ADD_PS KEY categories.category_id AS String
ADD_PS ATTRIBUTE category_id TO products WITH TYPE String
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id, category_id }
-- categories: { category_id, category_name, description }

-- ============================================
-- Step 9: FLATTEN address sub-objects in suppliers, customers, employees
-- ============================================
FLATTEN_PS suppliers.address
RENAME_PS ATTRIBUTE address_street TO street IN suppliers
RENAME_PS ATTRIBUTE address_city TO city IN suppliers
RENAME_PS ATTRIBUTE address_region TO region IN suppliers
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN suppliers
RENAME_PS ATTRIBUTE address_country TO country IN suppliers
FLATTEN_PS customers.address
RENAME_PS ATTRIBUTE address_street TO street IN customers
RENAME_PS ATTRIBUTE address_city TO city IN customers
RENAME_PS ATTRIBUTE address_region TO region IN customers
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN customers
RENAME_PS ATTRIBUTE address_country TO country IN customers
FLATTEN_PS employees.address
RENAME_PS ATTRIBUTE address_street TO street IN employees
RENAME_PS ATTRIBUTE address_city TO city IN employees
```

```
RENAME_PS ATTRIBUTE address_region TO region IN employees
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN employees
RENAME_PS ATTRIBUTE address_country TO country IN employees

-- FLATTEN orders.ship_destination (shipping address)
FLATTEN_PS orders.ship_destination
RENAME_PS ATTRIBUTE ship_destination_ship_address TO ship_address IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_city TO ship_city IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_region TO ship_region IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_postal_code TO ship_postal_code IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_country TO ship_country IN orders
-- Result:
-- suppliers: { supplier_id, company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- customers: { customer_id, company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- employees: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, street, city, postal_c

-- ============================================
-- Step 10: Add primary key for orders
-- ============================================
ADD_PS KEY orders.order_id AS String
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer_id, employee_id, shipper_id }

-- ============================================
-- Step 11: Convert FK fields to relationships (products)
-- ============================================
DELETE_PS ATTRIBUTE products.supplier_id
DELETE_PS ATTRIBUTE products.category_id
ADD_PS RELTYPE SUPPLIES BETWEEN suppliers AND products
ADD_PS RELTYPE PART_OF BETWEEN products AND categories
-- Result:
-- products node: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
-- Relationships: SUPPLIES (suppliers -> products), PART_OF (products -> categories)

-- ============================================
-- Step 12: Convert FK fields to relationships (orders)
-- ============================================
DELETE_PS ATTRIBUTE orders.customer_id
DELETE_PS ATTRIBUTE orders.employee_id
DELETE_PS ATTRIBUTE orders.shipper_id
ADD_PS RELTYPE PURCHASED BETWEEN customers AND orders
ADD_PS RELTYPE SOLD BETWEEN employees AND orders
ADD_PS RELTYPE SHIPPED_VIA BETWEEN orders AND shippers
-- Result:
-- orders node: { order_id, order_date, required_date, shipped_date, freight, ship_name, ship_city }
-- Relationships: PURCHASED, SOLD, SHIPPED_VIA

-- ============================================
-- Step 13: Convert reports_to field to self-relationship (employees)
-- ============================================
DELETE_PS ATTRIBUTE employees.reports_to
ADD_PS RELTYPE REPORTS_TO BETWEEN employees AND employees
-- Result:
-- employees node: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code, co
-- Relationship: REPORTS_TO (employees -> employees)

-- ============================================
-- Step 14: TRANSFORM order_details to CONTAINS relationship
-- ============================================
-- order_details has: order_id, product_id, unit_price, quantity, discount
-- Remove FK fields, then transform to relationship with remaining properties
DELETE_PS ATTRIBUTE order_details.order_id
DELETE_PS ATTRIBUTE order_details.product_id
TRANSFORM_PS order_details TO RELATIONSHIP BETWEEN orders AND products
RENAME_PS RELTYPE order_details TO CONTAINS
-- Result:
-- CONTAINS relationship (orders -> products) with properties: unit_price, quantity, discount

-- Final Result (7 nodes, 7 relationships):
-- categories:  { category_id [PK], category_name, description }
-- suppliers:   { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- shippers:    { shipper_id [PK], company_name, phone }
-- employees:   { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code,
-- customers:   { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- products:    { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
-- orders:      { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }
-- SUPPLIES (suppliers -> products), PART_OF (products -> categories)
-- PURCHASED (customers -> orders), SOLD (employees -> orders), SHIPPED_VIA (orders -> shippers)
-- REPORTS_TO (employees -> employees)
-- CONTAINS (orders -> products) with { unit_price, quantity, discount }
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- ============================================
-- Operation Summary (50 steps, 9 operation types):
--   UNNEST:            7    FLATTEN:           3
--   RENAME_ATTRIBUTE: 13    ADD_PRIMARY_KEY:   7
--   ADD_ATTRIBUTE:     7    DELETE_ATTRIBUTE: 11
--   ADD_RELTYPE:       7    TRANSFORM:         1
--   RENAME_RELTYPE:    1
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

12/16 northwind_neo4j_to_mongo.smel_ps

```
-- SMEL Northwind: Neo4j -> MongoDB (Specific Operations Version)
-- Graph (Nodes + Relationships) -> Document (Embedded 4-level nesting)
-- Direction: Graph (Nodes + Edges) -> Denormalized (Embedded)
--
-- Pipeline: Source Neo4j -> Reverse Engineering -> Meta V1 (Graph)
--           -> SMEL Operations (this script) -> Meta V2 (Document)
--           -> Forward Engineering -> Target MongoDB
--
-- Operation Semantics:
--   TRANSFORM:        Convert relationship to entity (or vice versa)
--   RENAME_ENTITY:    Rename an entity
--   ADD_ATTRIBUTE:    Add new attribute to entity
--   DELETE_RELTYPE:   Remove relationship type
--   UNFLATTEN:        Combine flat fields into nested object
--   NEST:             Embed separate entity into parent as nested object
--   RENAME_ATTRIBUTE: Rename a field within an entity
-- Note: Neo4j source already uses lowercase entity names (categories, suppliers, etc.)

MIGRATION northwind_neo4j_to_mongo:1.0
FROM GRAPH TO DOCUMENT
USING northwind_schema:1

-- ============================================
-- Source: Neo4j Graph (7 nodes, 7 relationships)
-- ============================================
-- Nodes:
--   categories (category_id, category_name, description)
--   suppliers (supplier_id, company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
--   shippers (shipper_id, company_name, phone)
--   employees (employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_code,
--   customers (customer_id, company_name, contact_name, contact_title, phone, fax, street, city, region, postal_code, country)
--   products (product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit)
--   orders (order_id, order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_pos
-- Relationships:
--   SUPPLIES (suppliers -> products), PART_OF (products -> categories)
--   PURCHASED (customers -> orders), SOLD (employees -> orders), SHIPPED_VIA (orders -> shippers)
--   REPORTS_TO (employees -> employees)
--   CONTAINS (orders -> products) with properties: unit_price, quantity, discount

-- ============================================
-- Target: MongoDB Orders Document
-- ============================================
-- {
--   _id: "ORD001",
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Step 1: TRANSFORM CONTAINS relationship -> order_details entity
-- ============================================
-- CONTAINS (orders -> products) with { unit_price, quantity, discount } becomes entity
-- Then rename from relationship name to table name, add FK columns.
TRANSFORM_PS CONTAINS TO ENTITY
RENAME_PS ENTITY CONTAINS TO order_details
ADD_PS ATTRIBUTE order_id TO order_details WITH TYPE String
ADD_PS ATTRIBUTE product_id TO order_details WITH TYPE String
-- Result:
-- order_details: { order_id, product_id, unit_price, quantity, discount }

-- ============================================
-- Step 2: Delete all relationships and add FK-like attributes
-- ============================================
-- REPORTS_TO -> add reports_to field to employees (self-reference)
DELETE_PS RELTYPE REPORTS_TO
ADD_PS ATTRIBUTE reports_to TO employees WITH TYPE String
-- SUPPLIES -> add supplier_id to products
DELETE_PS RELTYPE SUPPLIES
ADD_PS ATTRIBUTE supplier_id TO products WITH TYPE String
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- PART_OF -> add category_id to products
DELETE_PS RELTYPE PART_OF
ADD_PS ATTRIBUTE category_id TO products WITH TYPE String
-- PURCHASED -> add customer_id to orders
DELETE_PS RELTYPE PURCHASED
ADD_PS ATTRIBUTE customer_id TO orders WITH TYPE String
-- SOLD -> add employee_id to orders
DELETE_PS RELTYPE SOLD
ADD_PS ATTRIBUTE employee_id TO orders WITH TYPE String
-- SHIPPED_VIA -> add shipper_id to orders
DELETE_PS RELTYPE SHIPPED_VIA
ADD_PS ATTRIBUTE shipper_id TO orders WITH TYPE String
-- Result:
-- employees: { ..., reports_to }
-- products: { ..., supplier_id, category_id }
-- orders: { ..., customer_id, employee_id, shipper_id }

-- ============================================
-- Step 3: UNFLATTEN address fields (3 entities have flat address -> sub-objects)
-- ============================================
UNFLATTEN_PS suppliers:street, city, region, postal_code, country AS address
UNFLATTEN_PS customers:street, city, region, postal_code, country AS address
UNFLATTEN_PS employees:street, city, region, postal_code, country AS address
UNFLATTEN_PS orders:ship_address, ship_city, ship_region, ship_postal_code, ship_country AS ship_destination
-- Result:
-- suppliers: { supplier_id, company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, cou
-- customers: { customer_id, company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, cou
-- employees: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city,

-- ============================================
-- Step 4: NEST categories into products (deepest first -- Level 3)
-- ============================================
NEST_PS categories:category_name, description IN products.category WHERE products.category_id = categories.category_id WITH DELETI
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id,
--             category{category_name, description} }

-- ============================================
-- Step 5: NEST suppliers into products (Level 3 -- with nested address)
-- ============================================
NEST_PS suppliers:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country} IN pr
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             category{category_name, description},
--             supplier{company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country}

-- ============================================
-- Step 6: NEST products into order_details (Level 2)
-- ============================================
NEST_PS products:product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, category{category_name, description},
-- Result:
-- order_details: { order_id, unit_price, quantity, discount,
--                  product{product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                          category{category_name, description},
--                          supplier{company_name, contact_name, contact_title, phone, fax, address{...}}} }

-- ============================================
-- Step 7: NEST shippers into orders (Level 1)
-- ============================================
NEST_PS shippers:company_name, phone IN orders.shipper WHERE orders.shipper_id = shippers.shipper_id WITH DELETION
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer_id, employee_id, shipper{company_name, phone} }

-- ============================================
-- Step 8: NEST employees into orders (Level 1 -- with address + self-ref)
-- ============================================
NEST_PS employees:last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city, region, pos
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer_id, shipper{...},
--           employee{last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{...}} }

-- ============================================
-- Step 9: NEST customers into orders (Level 1 -- with address)
-- ============================================
NEST_PS customers:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country} IN or
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
```

```
--           shipper{...}, employee{...},
--           customer{company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country}}

-- ============================================
-- Step 10: NEST order_details into orders (Level 1 -- as array)
-- ============================================
NEST_PS order_details:unit_price, quantity, discount, product{product_name, unit_price, units_in_stock, discontinued, quantity_per
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer{...}, employee{...}, shipper{...},
--           details[{unit_price, quantity, discount, product{...}}] }

-- ============================================
-- Step 11: Rename PK for MongoDB convention
-- ============================================
RENAME_PS ATTRIBUTE order_id TO _id IN orders
-- Final Result:
-- orders: {
--   _id,
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Operation Summary (27 steps, 7 operation types):
--   TRANSFORM:         1    RENAME_ENTITY:     1
--   ADD_ATTRIBUTE:      8    DELETE_RELTYPE:    6
--   UNFLATTEN:          3    NEST:              7
--   RENAME_ATTRIBUTE:   1
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

13/16 northwind_mongo_to_cass.smel_ps

```
-- SMEL Northwind: MongoDB -> Cassandra (Cross-model Migration)
-- Orders Document (multi-level nested objects + arrays) -> Wide-column Tables
-- Direction: Denormalized (Embedded) -> Columnar (query-driven partition/clustering keys)
--
-- Pipeline: Source MongoDB -> Reverse Engineering -> Meta V1 (Document)
--           -> SMEL Operations (this script) -> Meta V2 (Columnar)
--           -> Forward Engineering -> Target Cassandra
--
-- Operation Semantics:
--   UNNEST:             Extract nested object to separate entity (normalization)
--   FLATTEN:            Flatten nested object fields into same entity (reduce depth by 1)
--   ADD_PARTITION_KEY:  Define partition key for query-driven data distribution
--   ADD_CLUSTERING_KEY: Define clustering key for sort order within partition

MIGRATION northwind_mongo_to_cass:1.0
FROM DOCUMENT TO COLUMNAR
USING northwind_schema:1

-- ============================================
-- Source: MongoDB Orders Document
-- ============================================
-- {
--   _id: "ORD001",
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Target: Cassandra Wide-column Tables (8 tables)
-- ============================================
-- categories     (category_id [PARTITION], category_name, description)
-- suppliers      (supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- shippers       (shipper_id [PARTITION], company_name, phone)
-- employees      (employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, regio
-- customers      (customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- products       (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_s
-- orders         (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight,
-- order_details  (order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount)

-- ============================================
-- Step 1: Rename PK from MongoDB convention
-- ============================================
-- Reverse of: RENAME_ATTRIBUTE order_id TO _id IN orders
RENAME_PS ATTRIBUTE _id TO order_id IN orders
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer{...}, employee{...}, shipper{...}, details[{...}] }

-- ============================================
-- Step 2: UNNEST details[] from orders -> order_details (Level 1 -- array)
-- ============================================
-- Reverse of: NEST order_details:... IN orders.details
UNNEST_PS orders.details:unit_price, quantity, discount, product{product_name, unit_price, units_in_stock, discontinued, quantity_
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer{...}, employee{...}, shipper{...} }
-- order_details: { order_id, unit_price, quantity, discount,
--                  product{product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                          category{...}, supplier{...}} }

-- ============================================
-- Step 3: UNNEST customer from orders -> customers (Level 1)
-- ============================================
-- Reverse of: NEST customers:... IN orders.customer
UNNEST_PS orders.customer:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, countr
DELETE_PS ATTRIBUTE customers.order_id
ADD_PS KEY customers.customer_id AS String
```

```
ADD_PS ATTRIBUTE customer_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id, ..., customer_id, employee{...}, shipper{...} }
-- customers: { customer_id, company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, cou

-- ============================================
-- Step 4: UNNEST employee from orders -> employees (Level 1)
-- ============================================
-- Reverse of: NEST employees:... IN orders.employee
UNNEST_PS orders.employee:last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city, reg
DELETE_PS ATTRIBUTE employees.order_id
ADD_PS KEY employees.employee_id AS String
ADD_PS ATTRIBUTE employee_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id, ..., customer_id, employee_id, shipper{...} }
-- employees: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{...} }

-- ============================================
-- Step 5: UNNEST shipper from orders -> shippers (Level 1)
-- ============================================
-- Reverse of: NEST shippers:... IN orders.shipper
UNNEST_PS orders.shipper:company_name, phone AS shippers WITH orders.order_id TO shippers.order_id
DELETE_PS ATTRIBUTE shippers.order_id
ADD_PS KEY shippers.shipper_id AS String
ADD_PS ATTRIBUTE shipper_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer_id, employee_id, shipper_id }
-- shippers: { shipper_id, company_name, phone }

-- ============================================
-- Step 6: UNNEST product from order_details -> products (Level 2)
-- ============================================
-- Reverse of: NEST products:... IN order_details.product
UNNEST_PS order_details.product:product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, category{category_name,
DELETE_PS ATTRIBUTE products.order_id
ADD_PS KEY products.product_id AS String
ADD_PS ATTRIBUTE product_id TO order_details WITH TYPE String
-- Result:
-- order_details: { order_id, product_id, unit_price, quantity, discount }
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             category{category_name, description}, supplier{...} }

-- ============================================
-- Step 7: UNNEST supplier from products -> suppliers (Level 3)
-- ============================================
-- Reverse of: NEST suppliers:... IN products.supplier
UNNEST_PS products.supplier:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, coun
DELETE_PS ATTRIBUTE suppliers.product_id
ADD_PS KEY suppliers.supplier_id AS String
ADD_PS ATTRIBUTE supplier_id TO products WITH TYPE String
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             supplier_id, category{category_name, description} }
-- suppliers: { supplier_id, company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, cou

-- ============================================
-- Step 8: UNNEST category from products -> categories (Level 3)
-- ============================================
-- Reverse of: NEST categories:... IN products.category
UNNEST_PS products.category:category_name, description AS categories WITH products.product_id TO categories.product_id
DELETE_PS ATTRIBUTE categories.product_id
ADD_PS KEY categories.category_id AS String
ADD_PS ATTRIBUTE category_id TO products WITH TYPE String
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             supplier_id, category_id }
-- categories: { category_id, category_name, description }

-- ============================================
-- Step 9: FLATTEN address sub-objects (3 entities -> flat fields)
-- ============================================
-- Reverse of: UNFLATTEN suppliers/customers/employees:street, city, postal_code, country AS address

-- 9a: FLATTEN suppliers.address
FLATTEN_PS suppliers.address
RENAME_PS ATTRIBUTE address_street TO street IN suppliers
RENAME_PS ATTRIBUTE address_city TO city IN suppliers
RENAME_PS ATTRIBUTE address_region TO region IN suppliers
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN suppliers
RENAME_PS ATTRIBUTE address_country TO country IN suppliers
```

```
-- Result:
-- suppliers: { supplier_id, company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }

-- 9b: FLATTEN customers.address
FLATTEN_PS customers.address
RENAME_PS ATTRIBUTE address_street TO street IN customers
RENAME_PS ATTRIBUTE address_city TO city IN customers
RENAME_PS ATTRIBUTE address_region TO region IN customers
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN customers
RENAME_PS ATTRIBUTE address_country TO country IN customers
-- Result:
-- customers: { customer_id, company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }

-- 9c: FLATTEN employees.address
FLATTEN_PS employees.address
RENAME_PS ATTRIBUTE address_street TO street IN employees
RENAME_PS ATTRIBUTE address_city TO city IN employees
RENAME_PS ATTRIBUTE address_region TO region IN employees
RENAME_PS ATTRIBUTE address_postal_code TO postal_code IN employees
RENAME_PS ATTRIBUTE address_country TO country IN employees

-- FLATTEN orders.ship_destination (shipping address)
FLATTEN_PS orders.ship_destination
RENAME_PS ATTRIBUTE ship_destination_ship_address TO ship_address IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_city TO ship_city IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_region TO ship_region IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_postal_code TO ship_postal_code IN orders
RENAME_PS ATTRIBUTE ship_destination_ship_country TO ship_country IN orders
-- Result:
-- employees: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, region, postal_code,

-- ============================================
-- Step 10: Restructure keys for Cassandra (complex tables)
-- ============================================
-- Simple tables (categories, suppliers, shippers, customers, employees) have simple PKs
-- that auto-convert to PARTITION KEYs via EntityKind auto-conversion.
-- Complex tables need explicit key restructuring.

-- 10a: Restructure products keys (PK -> partition + clustering)
-- Cassandra: partition by category_id (query products by category), cluster by product_id
DELETE_PS PRIMARY KEY product_id FROM products
ADD_PS PARTITION KEY (category_id, supplier_id) TO products
ADD_PS CLUSTERING KEY product_id TO products
-- Result:
-- products (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_stock,

-- 10b: Restructure orders keys (PK -> partition + clustering)
-- Cassandra: partition by customer_id (query orders by customer), cluster by order_date, order_id
DELETE_PS PRIMARY KEY order_id FROM orders
ADD_PS PARTITION KEY customer_id TO orders
ADD_PS CLUSTERING KEY order_date TO orders
ADD_PS CLUSTERING KEY order_id TO orders
-- Result:
-- orders (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight, ship_nam

-- 10c: Restructure order_details keys (add partition + clustering)
-- order_details has order_id and product_id as regular attributes (from UNNEST carry-field)
-- Cassandra: partition by order_id (query details by order), cluster by product_id
ADD_PS PARTITION KEY (order_id, product_id) TO order_details
-- Result:
-- order_details (order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount)

-- ============================================
-- Final Target (8 wide-column tables):
-- categories:     { category_id [PARTITION], category_name, description }
-- suppliers:      { supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, co
-- shippers:       { shipper_id [PARTITION], company_name, phone }
-- employees:      { employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, reg
-- customers:      { customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, co
-- products:       { category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in
-- orders:         { customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight
-- order_details:  { order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount }
-- ============================================

-- ============================================
-- Operation Summary (44 steps):
--   RENAME_ATTRIBUTE: 13  (1 PK rename + 12 FLATTEN renames)
--   UNNEST: 7             (details, customer, employee, shipper, product, supplier, category)
--   FLATTEN: 3            (suppliers.address, customers.address, employees.address)
--   ADD_PRIMARY_KEY: 6    (customers, employees, shippers, products, suppliers, categories)
--   DELETE_ATTRIBUTE: 6   (carry-field cleanup for customers, employees, shippers, products, suppliers, categories)
--   ADD_ATTRIBUTE: 6      (FK fields: customer_id, employee_id, shipper_id, product_id, supplier_id, category_id)
```

```
--   DELETE_PRIMARY_KEY: 2  (products, orders -- before Cassandra key restructuring)
--   ADD_PARTITION_KEY: 3   (products, orders, order_details)
--   ADD_CLUSTERING_KEY: 3  (products, orders x2)
--   EntityKind auto-convert: DOCUMENT -> WIDE_COLUMN_TABLE (all 8 entities)
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

14/16 northwind_cass_to_mongo.smel_ps

```
-- SMEL Northwind: Cassandra -> MongoDB (Cross-model Migration)
-- Wide-column Tables -> Orders Document (multi-level nested objects + arrays)
-- Direction: Columnar (query-driven partition/clustering keys) -> Denormalized (Embedded)
--
-- Pipeline: Source Cassandra -> Reverse Engineering -> Meta V1 (Columnar)
--           -> SMEL Operations (this script) -> Meta V2 (Document)
--           -> Forward Engineering -> Target MongoDB
--
-- Operation Semantics:
--   DELETE_PARTITION_KEY:   Remove Cassandra partition key
--   DELETE_CLUSTERING_KEY:  Remove Cassandra clustering key
--   UNFLATTEN:              Combine flat fields into nested object (reverse of FLATTEN)
--   NEST:                   Embed separate entity into parent as nested object (reverse of UNNEST)

MIGRATION northwind_cass_to_mongo:1.0
FROM COLUMNAR TO DOCUMENT
USING northwind_schema:1

-- ============================================
-- Source: Cassandra Wide-column Tables (8 tables)
-- ============================================
-- categories     (category_id [PARTITION], category_name, description)
-- suppliers      (supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- shippers       (shipper_id [PARTITION], company_name, phone)
-- employees      (employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, regio
-- customers      (customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- products       (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_s
-- orders         (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight,
-- order_details  (order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount)

-- ============================================
-- Target: MongoDB Orders Document
-- ============================================
-- {
--   _id: "ORD001",
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Step 1: Remove Cassandra key structure for complex tables
-- ============================================
-- Simple tables (categories, suppliers, shippers, customers, employees) have simple
-- partition keys that auto-convert via EntityKind auto-conversion.
-- Complex tables need explicit key removal.

-- 1a: Remove products keys (partition + clustering)
DELETE_PS PARTITION KEY (category_id, supplier_id) FROM products
DELETE_PS CLUSTERING KEY product_id FROM products
-- Result:
-- products (product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id, category_id)

-- 1b: Remove orders keys (partition + clustering x2)
DELETE_PS PARTITION KEY customer_id FROM orders
DELETE_PS CLUSTERING KEY order_date FROM orders
DELETE_PS CLUSTERING KEY order_id FROM orders
-- Result:
-- orders (order_id, order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, ship_posta

-- 1c: Remove order_details keys (partition + clustering)
DELETE_PS PARTITION KEY (order_id, product_id) FROM order_details
-- Result:
-- order_details (order_id, product_id, unit_price, quantity, discount)

-- ============================================
-- Step 2: UNFLATTEN address fields (3 entities have flat address -> sub-objects)
-- ============================================
-- Reverse of: FLATTEN suppliers.address / customers.address / employees.address
UNFLATTEN_PS suppliers:street, city, region, postal_code, country AS address
```

Northwind SMEL Scripts (Pauschalisiert)

```
UNFLATTEN_PS customers:street, city, region, postal_code, country AS address
UNFLATTEN_PS employees:street, city, region, postal_code, country AS address
UNFLATTEN_PS orders:ship_address, ship_city, ship_region, ship_postal_code, ship_country AS ship_destination
-- Result:
-- suppliers: { supplier_id, company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, cou
-- customers: { customer_id, company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, cou
-- employees: { employee_id, last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city,

-- ============================================
-- Step 3: NEST categories into products (Level 3 -- deepest embedding first)
-- ============================================
-- Reverse of: UNNEST products.category:category_name, description AS categories
NEST_PS categories:category_name, description IN products.category WHERE products.category_id = categories.category_id WITH DELETI
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id,
--             category{category_name, description} }

-- ============================================
-- Step 4: NEST suppliers into products (Level 3 -- with nested address)
-- ============================================
-- Reverse of: UNNEST products.supplier:company_name, contact_name, contact_title, phone, fax, address{...} AS suppliers
NEST_PS suppliers:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country} IN pr
-- Result:
-- products: { product_id, product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--             category{category_name, description},
--             supplier{company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country}

-- ============================================
-- Step 5: NEST products into order_details (Level 2)
-- ============================================
-- Reverse of: UNNEST order_details.product:product_name, unit_price, ... AS products
NEST_PS products:product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, category{category_name, description},
-- Result:
-- order_details: { order_id, unit_price, quantity, discount,
--                  product{product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                          category{category_name, description},
--                          supplier{company_name, contact_name, contact_title, phone, fax, address{...}}} }

-- ============================================
-- Step 6: NEST shippers into orders (Level 1)
-- ============================================
-- Reverse of: UNNEST orders.shipper:company_name, phone AS shippers
NEST_PS shippers:company_name, phone IN orders.shipper WHERE orders.shipper_id = shippers.shipper_id WITH DELETION
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer_id, employee_id, shipper{company_name, phone} }

-- ============================================
-- Step 7: NEST employees into orders (Level 1 -- with address)
-- ============================================
-- Reverse of: UNNEST orders.employee:last_name, first_name, ... AS employees
NEST_PS employees:last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{street, city, region, pos
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer_id, shipper{...},
--           employee{last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to, address{...}} }

-- ============================================
-- Step 8: NEST customers into orders (Level 1 -- with address)
-- ============================================
-- Reverse of: UNNEST orders.customer:company_name, contact_name, ... AS customers
NEST_PS customers:company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country} IN or
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           shipper{...}, employee{...},
--           customer{company_name, contact_name, contact_title, phone, fax, address{street, city, region, postal_code, country}}

-- ============================================
-- Step 9: NEST order_details into orders (Level 1 -- as array)
-- ============================================
-- Reverse of: UNNEST orders.details:unit_price, quantity, discount, product{...} AS order_details
NEST_PS order_details:unit_price, quantity, discount, product{product_name, unit_price, units_in_stock, discontinued, quantity_per
-- Result:
-- orders: { order_id, order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--           customer{...}, employee{...}, shipper{...},
--           details[{unit_price, quantity, discount, product{...}}] }

-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- Step 10: Rename PK for MongoDB convention
-- ============================================
-- Reverse of: RENAME_ATTRIBUTE _id TO order_id IN orders
RENAME_PS ATTRIBUTE order_id TO _id IN orders
ADD_PS KEY _id TO orders
-- Final Result:
-- orders: {
--   _id,
--   order_date, required_date, shipped_date, freight, ship_name,
--   ship_destination: { ship_address, ship_city, ship_region, ship_postal_code, ship_country },
--   customer: { company_name, contact_name, contact_title, phone, fax,
--               address: { street, city, region, postal_code, country } },
--   employee: { last_name, first_name, title, birth_date, hire_date, phone, notes, reports_to,
--               address: { street, city, region, postal_code, country } },
--   shipper: { company_name, phone },
--   details: [
--     { unit_price, quantity, discount,
--       product: { product_name, unit_price, units_in_stock, discontinued, quantity_per_unit,
--                  category: { category_name, description },
--                  supplier: { company_name, contact_name, contact_title, phone, fax,
--                              address: { street, city, region, postal_code, country } } } }
--   ]
-- }

-- ============================================
-- Operation Summary (19 steps):
--   DELETE_PARTITION_KEY:  3   (products, orders, order_details)
--   DELETE_CLUSTERING_KEY: 3   (products, orders x2)
--   UNFLATTEN: 3               (address sub-objects for suppliers, customers, employees)
--   NEST: 7                    (categories->products, suppliers->products, products->order_details,
--                               shippers->orders, employees->orders, customers->orders, order_details->orders)
--   RENAME_ATTRIBUTE: 1        (order_id -> _id)
--   ADD_PS KEY: 1              (_id for orders -- lost after Cassandra key removal)
--   EntityKind auto-convert: WIDE_COLUMN_TABLE -> DOCUMENT (all 8 entities)
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

15/16 northwind_neo4j_to_cass.smel_ps

```
-- SMEL Northwind: Neo4j -> Cassandra (Specific Operations Version)
-- Cross-model Schema Migration: Graph -> Columnar
-- Direction: northwind_neo4j.cypher -> northwind_cassandra.cql
--
-- Pipeline: Source Neo4j -> Reverse Engineering -> Meta V1 (Graph)
--           -> SMEL Operations (this script) -> Meta V2 (Columnar)
--           -> Forward Engineering -> Target Cassandra
--
-- Operation Semantics:
--   TRANSFORM:          Convert relationship (graph edge) to entity (wide-column table)
--   RENAME_ENTITY:      Rename entity after TRANSFORM (relationship name -> table name)
--   ADD_ATTRIBUTE:      Add FK-like attribute to entity (graph edge replaced by column)
--   DELETE_RELTYPE:     Remove relationship type (graph edge no longer needed)
--   DELETE_PRIMARY_KEY: Remove graph PK before adding partition/clustering keys
--   ADD_PARTITION_KEY:  Define partition key for query-driven data distribution
--   ADD_CLUSTERING_KEY: Define clustering key for sort order within partition
-- Note: Neo4j source already uses lowercase entity names (categories, suppliers, etc.)

MIGRATION northwind_neo4j_to_cass:1.0
FROM GRAPH TO COLUMNAR
USING northwind_schema:1

-- ============================================
-- Source: Neo4j Graph (7 nodes, 7 relationships)
-- ============================================
-- Nodes:
--   categories:  { category_id [PK], category_name, description }
--   suppliers:   { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
--   shippers:    { shipper_id [PK], company_name, phone }
--   employees:   { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code
--   customers:   { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
--   products:    { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
--   orders:      { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }
-- Relationships:
--   REPORTS_TO  (employees -> employees)
--   SUPPLIES    (suppliers -> products)
--   PART_OF     (products -> categories)
--   PURCHASED   (customers -> orders)
--   SOLD        (employees -> orders)
--   SHIPPED_VIA (orders -> shippers)
--   CONTAINS    (orders -> products) with properties { unit_price, quantity, discount }

-- ============================================
-- Target: Cassandra Wide-column Tables (8 tables)
-- ============================================
-- categories     (category_id [PARTITION], category_name, description)
-- suppliers      (supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- shippers       (shipper_id [PARTITION], company_name, phone)
-- employees      (employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, regio
-- customers      (customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- products       (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_s
-- orders         (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight,
-- order_details  (order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount)

-- ============================================
-- Step 1: CONTAINS relationship -> order_details entity
-- ============================================
-- The CONTAINS relationship carries properties (unit_price, quantity, discount).
-- TRANSFORM converts it to an entity, preserving those properties.
-- Then rename from relationship name to table name, add FK-like columns.
TRANSFORM_PS CONTAINS TO ENTITY
RENAME_PS ENTITY CONTAINS TO order_details
ADD_PS ATTRIBUTE order_id TO order_details WITH TYPE String
ADD_PS ATTRIBUTE product_id TO order_details WITH TYPE String
-- Result:
-- order_details: { order_id, product_id, unit_price, quantity, discount }

-- ============================================
-- Step 2: Delete relationships and add FK-like attributes
-- ============================================
-- Cassandra has no FK constraints, so relationships become plain columns.

-- employees self-reference: REPORTS_TO -> reports_to column
DELETE_PS RELTYPE REPORTS_TO
ADD_PS ATTRIBUTE reports_to TO employees WITH TYPE String
-- Result:
-- employees: { employee_id [PK], ..., reports_to }

-- products <- supplier: SUPPLIES -> supplier_id column
DELETE_PS RELTYPE SUPPLIES
ADD_PS ATTRIBUTE supplier_id TO products WITH TYPE String
```

```
-- Result:
-- products: { product_id [PK], ..., supplier_id }

-- products -> category: PART_OF -> category_id column
DELETE_PS RELTYPE PART_OF
ADD_PS ATTRIBUTE category_id TO products WITH TYPE String
-- Result:
-- products: { product_id [PK], ..., supplier_id, category_id }

-- orders <- customer: PURCHASED -> customer_id column
DELETE_PS RELTYPE PURCHASED
ADD_PS ATTRIBUTE customer_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id [PK], ..., customer_id }

-- orders <- employee: SOLD -> employee_id column
DELETE_PS RELTYPE SOLD
ADD_PS ATTRIBUTE employee_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id [PK], ..., customer_id, employee_id }

-- orders -> shipper: SHIPPED_VIA -> shipper_id column
DELETE_PS RELTYPE SHIPPED_VIA
ADD_PS ATTRIBUTE shipper_id TO orders WITH TYPE String
-- Result:
-- orders: { order_id [PK], ..., customer_id, employee_id, shipper_id }

-- ============================================
-- Step 3: Restructure keys for Cassandra (complex tables)
-- ============================================

-- products: partition by category_id, cluster by product_id
DELETE_PS PRIMARY KEY product_id FROM products
ADD_PS PARTITION KEY (category_id, supplier_id) TO products
ADD_PS CLUSTERING KEY product_id TO products
-- Result:
-- products: { category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_stock

-- orders: partition by customer_id, cluster by order_date + order_id
DELETE_PS PRIMARY KEY order_id FROM orders
ADD_PS PARTITION KEY customer_id TO orders
ADD_PS CLUSTERING KEY order_date TO orders
ADD_PS CLUSTERING KEY order_id TO orders
-- Result:
-- orders: { customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight, ship_n

-- order_details: partition by order_id, cluster by product_id
-- (no existing PK to remove -- just created by TRANSFORM)
ADD_PS PARTITION KEY (order_id, product_id) TO order_details
-- Result:
-- order_details: { order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount }

-- ============================================
-- Simple tables: categories, suppliers, shippers, employees, customers
-- ============================================
-- These nodes have simple PKs that auto-convert to PARTITION KEYs
-- via EntityKind auto-conversion (VERTEX -> WIDE_COLUMN_TABLE).
-- No explicit SMEL operations needed.

-- ============================================
-- Final Target (8 wide-column tables):
-- categories:     { category_id [PARTITION], category_name, description }
-- suppliers:      { supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, co
-- shippers:       { shipper_id [PARTITION], company_name, phone }
-- employees:      { employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, reg
-- customers:      { customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, co
-- products:       { category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in
-- orders:         { customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight
-- order_details:  { order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount }
-- ============================================

-- ============================================
-- Operation Summary (25 steps, 7 operation types):
--   RENAME_ENTITY:      1    TRANSFORM:          1
--   ADD_ATTRIBUTE:      8    DELETE_RELTYPE:     6
--   DELETE_PRIMARY_KEY: 2    ADD_PARTITION_KEY:  3
--   ADD_CLUSTERING_KEY: 3
--   EntityKind auto-convert: VERTEX -> WIDE_COLUMN_TABLE (all 8 entities)
-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

16/16 northwind_cass_to_neo4j.smel_ps

```
-- SMEL Northwind: Cassandra -> Neo4j (Specific Operations Version)
-- Cross-model Schema Migration: Columnar -> Graph
-- Direction: northwind_cassandra.cql -> northwind_neo4j.cypher
--
-- Pipeline: Source Cassandra -> Reverse Engineering -> Meta V1 (Columnar)
--           -> SMEL Operations (this script) -> Meta V2 (Graph)
--           -> Forward Engineering -> Target Neo4j
--
-- Operation Semantics:
--   DELETE_PARTITION_KEY:  Remove Cassandra partition key before restructuring
--   DELETE_CLUSTERING_KEY: Remove Cassandra clustering key
--   ADD_PRIMARY_KEY:       Add primary key to entity (graph node PK)
--   DELETE_ATTRIBUTE:      Remove FK-like attribute (replaced by graph edge)
--   ADD_RELTYPE:           Add relationship type between nodes (graph edge)
--   TRANSFORM:             Convert entity (wide-column table) to relationship (graph edge)
--   RENAME_RELTYPE:        Rename relationship type after TRANSFORM

MIGRATION northwind_cass_to_neo4j:1.0
FROM COLUMNAR TO GRAPH
USING northwind_schema:1

-- ============================================
-- Source: Cassandra Wide-column Tables (8 tables)
-- ============================================
-- categories     (category_id [PARTITION], category_name, description)
-- suppliers      (supplier_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- shippers       (shipper_id [PARTITION], company_name, phone)
-- employees      (employee_id [PARTITION], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, regio
-- customers      (customer_id [PARTITION], company_name, contact_name, contact_title, phone, fax, street, city, region, postal_co
-- products       (category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING], product_name, unit_price, units_in_s
-- orders         (customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING], required_date, shipped_date, freight,
-- order_details  (order_id [PARTITION], product_id [PARTITION], unit_price, quantity, discount)

-- ============================================
-- Target: Neo4j Graph (7 nodes, 7 relationships)
-- ============================================
-- Nodes:
--   categories:  { category_id [PK], category_name, description }
--   suppliers:   { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
--   shippers:    { shipper_id [PK], company_name, phone }
--   employees:   { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code
--   customers:   { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
--   products:    { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
--   orders:      { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }
-- Relationships:
--   REPORTS_TO  (employees -> employees)
--   SUPPLIES    (suppliers -> products)
--   PART_OF     (products -> categories)
--   PURCHASED   (customers -> orders)
--   SOLD        (employees -> orders)
--   SHIPPED_VIA (orders -> shippers)
--   CONTAINS    (orders -> products) with properties { unit_price, quantity, discount }

-- ============================================
-- Step 1: Restructure keys - remove Cassandra partition/clustering for complex tables
-- ============================================

-- products: partition + clustering -> simple PK
-- Cassandra: category_id [PARTITION], supplier_id [PARTITION], product_id [CLUSTERING]
-- Graph: product_id [PK]
DELETE_PS PARTITION KEY (category_id, supplier_id) FROM products
DELETE_PS CLUSTERING KEY product_id FROM products
ADD_PS KEY product_id TO products
-- Result:
-- products: { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, supplier_id, category_i

-- orders: partition + clustering -> simple PK
-- Cassandra: customer_id [PARTITION], order_date [CLUSTERING], order_id [CLUSTERING]
-- Graph: order_id [PK]
DELETE_PS PARTITION KEY customer_id FROM orders
DELETE_PS CLUSTERING KEY order_date FROM orders
DELETE_PS CLUSTERING KEY order_id FROM orders
ADD_PS KEY order_id TO orders
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, shi

-- order_details: remove partition + clustering (no PK added -- will become relationship)
-- Cassandra: order_id [PARTITION], product_id [PARTITION]
DELETE_PS PARTITION KEY (order_id, product_id) FROM order_details
-- Result:
-- order_details: { order_id, product_id, unit_price, quantity, discount }
```

```
-- ============================================
-- Step 2: Delete FK-like attributes and add relationships
-- ============================================

-- employees self-reference: reports_to column -> REPORTS_TO relationship
DELETE_PS ATTRIBUTE employees.reports_to
ADD_PS RELTYPE REPORTS_TO BETWEEN employees AND employees
-- Result:
-- employees: { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code, co
-- + REPORTS_TO relationship (employees -> employees)

-- products -> supplier: supplier_id column -> SUPPLIES relationship
DELETE_PS ATTRIBUTE products.supplier_id
ADD_PS RELTYPE SUPPLIES BETWEEN suppliers AND products
-- Result:
-- products: { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit, category_id }
-- + SUPPLIES relationship (suppliers -> products)

-- products -> category: category_id column -> PART_OF relationship
DELETE_PS ATTRIBUTE products.category_id
ADD_PS RELTYPE PART_OF BETWEEN products AND categories
-- Result:
-- products: { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }

-- orders -> customer: customer_id column -> PURCHASED relationship
DELETE_PS ATTRIBUTE orders.customer_id
ADD_PS RELTYPE PURCHASED BETWEEN customers AND orders
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, shi
-- + PURCHASED relationship (customers -> orders)

-- orders -> employee: employee_id column -> SOLD relationship
DELETE_PS ATTRIBUTE orders.employee_id
ADD_PS RELTYPE SOLD BETWEEN employees AND orders
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_address, ship_city, ship_region, shi
-- + SOLD relationship (employees -> orders)

-- orders -> shipper: shipper_id column -> SHIPPED_VIA relationship
DELETE_PS ATTRIBUTE orders.shipper_id
ADD_PS RELTYPE SHIPPED_VIA BETWEEN orders AND shippers
-- Result:
-- orders: { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }

-- ============================================
-- Step 3: TRANSFORM order_details to CONTAINS relationship
-- ============================================
-- order_details has: order_id, product_id, unit_price, quantity, discount
-- Remove FK-like columns, then transform entity to relationship.
-- Remaining attributes (unit_price, quantity, discount) become relationship properties.
DELETE_PS ATTRIBUTE order_details.order_id
DELETE_PS ATTRIBUTE order_details.product_id
TRANSFORM_PS order_details TO RELATIONSHIP BETWEEN orders AND products
RENAME_PS RELTYPE order_details TO CONTAINS
-- Result:
-- order_details table removed; CONTAINS relationship created:
-- CONTAINS (orders -> products) with properties: { unit_price, quantity, discount }
-- Note: TRANSFORM creates relationship with entity name "order_details",
--       then RENAME_RELTYPE renames it to match Neo4j convention "CONTAINS".

-- ============================================
-- Final Meta V2 (7 nodes, 7 relationship types):
-- ============================================
-- categories:  { category_id [PK], category_name, description }
-- suppliers:   { supplier_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- shippers:    { shipper_id [PK], company_name, phone }
-- employees:   { employee_id [PK], last_name, first_name, title, birth_date, hire_date, phone, notes, street, city, postal_code,
-- customers:   { customer_id [PK], company_name, contact_name, contact_title, phone, fax, street, city, postal_code, country }
-- products:    { product_id [PK], product_name, unit_price, units_in_stock, discontinued, quantity_per_unit }
-- orders:      { order_id [PK], order_date, required_date, shipped_date, freight, ship_name, ship_city }
-- REPORTS_TO  (employees -> employees)
-- SUPPLIES    (suppliers -> products)
-- PART_OF     (products -> categories)
-- PURCHASED   (customers -> orders)
-- SOLD        (employees -> orders)
-- SHIPPED_VIA (orders -> shippers)
-- CONTAINS    (orders -> products) with properties { unit_price, quantity, discount }
-- [order_details TRANSFORMED to CONTAINS relationship]
-- ============================================

-- ============================================
```

Northwind SMEL Scripts (Pauschalisiert)

```
-- Operation Summary (25 steps, 7 operation types):
--   DELETE_PARTITION_KEY:  3    DELETE_CLUSTERING_KEY: 3
--   ADD_PRIMARY_KEY:       2    DELETE_ATTRIBUTE:      8
--   ADD_RELTYPE:           6    TRANSFORM:             1
--   RENAME_RELTYPE:        1
--   EntityKind auto-convert: WIDE_COLUMN_TABLE -> VERTEX (all 8 entities)
-- ============================================
```